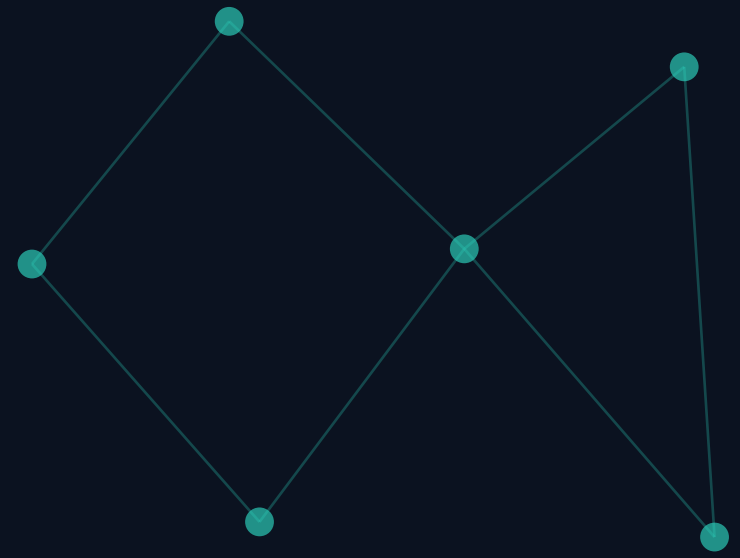
CLASE 8 · UNIDAD 4 · SEMANA 8

Retrieval-Augmented Generation



Darle al modelo una fuente que no está en sus pesos —
y aprender a medir si sirvió.

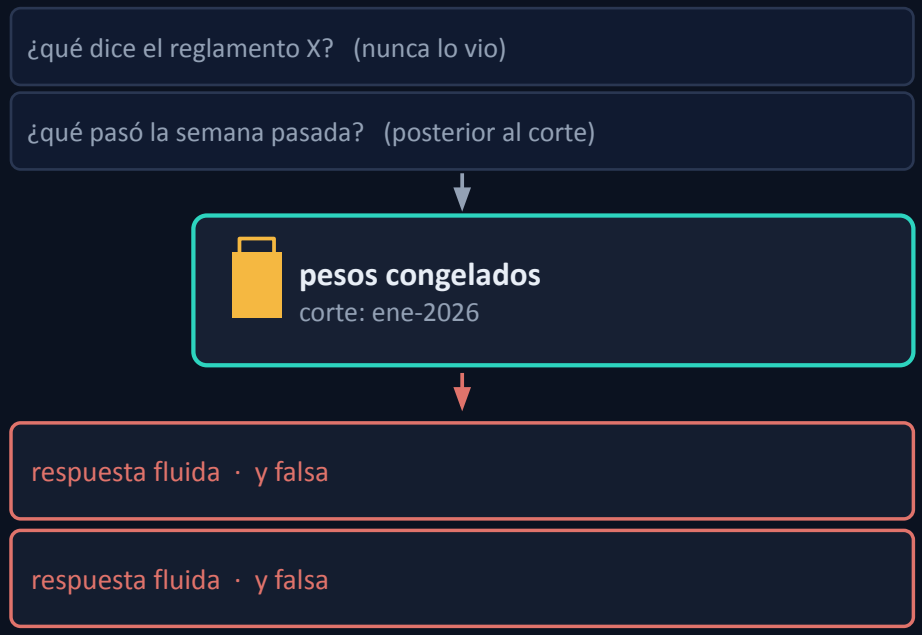
IA Generativa Avanzada y Sistemas Multi-Agente · Licenciatura en Tecnología Digital · Universidad Torcuato Di Tella

El modelo está congelado — y por eso inventa

Dónde quedamos. Cuando tratamos alineamiento y evaluación cerramos con la peor debilidad práctica del modelo alineado: inventa. Y dio el mecanismo: el pretraining premia continuaciones probables, no verdaderas. El modelo aprende la forma de una respuesta correcta aunque no tenga el hecho, y no hay señal interna de “no sé”.

No es un accidente. Alucinar es lo que hace un modelo entrenado para continuar texto cuando no sabe. El alineamiento lo suaviza en modales, pero puede empeorar la confianza con la que lo dice.

Y hay un segundo hueco, del mismo origen. Los pesos se congelaron el día que terminó el entrenamiento. El modelo no conoce los datos propios —nunca los vio— ni lo reciente —pasó después—. Ante una pregunta por el reglamento de una empresa o por algo de la semana pasada, no tiene de dónde sacarlo, y aun así contesta.



fluidez ≠ verdad: el objetivo era continuar, no acertar

Los dos huecos son el mismo hueco. No conoce lo propio, no conoce lo nuevo, y no tiene cómo avisar. Lo que falta no es más entrenamiento: es una fuente.

Tres vías, no dos

Reentrenar. Fine-tunear con los datos propios. Sirve poco: el fine-tuning ajusta forma y comportamiento, no incorpora hechos de forma confiable. Peor: forzar conocimiento nuevo aumenta la tasa de alucinación. Y mañana hay un dato nuevo y hay que entrenar de nuevo.

Meter todo en el prompt. Si el corpus entra en la ventana, se pega y listo — muchas veces es lo correcto. El límite no es sólo si entra: como vimos en prompt engineering, la ventana no se usa pareja y que la confiabilidad cae a medida que el input crece.

Recuperar y entregar. Buscar en una fuente externa los pasajes que la pregunta necesita y pasárselos como contexto. Eso es RAG (retrieval-augmented generation). No toca los pesos: cambia el input.

El criterio. RAG se elige cuando el corpus no entra, cambia seguido, o hace falta trazabilidad. Si no se cumple ninguna, no armes un pipeline: pegá el documento.



La pregunta no es cuál es mejor: es cuál corresponde. Las tres siguen vivas en 2026, y elegir mal es el error de arquitectura más común. Hoy construimos la tercera — pero decidir no usarla es parte de saber usarla.

Lo que RAG sí hace, y lo que no

La promesa, con su letra chica. Ya quedó anticipado que RAG es la mitigación más importante de las alucinaciones. Es cierto, con un matiz: RAG no elimina la alucinación. La reduce y, sobre todo, la vuelve verificable.

Por qué eso es más, y no menos. Cuando tratamos alineamiento y evaluación, detectar una invención requería trucos indirectos —muestrear varias veces y ver si divergían— porque no había con qué comparar. Ahora sí: está el pasaje que le dimos. Se puede chequear la respuesta contra él.

Y de ahí sale la pregunta que atraviesa la clase. Si el modelo va a responder con el texto que le entregamos, todo el problema se corre a entregarle el texto correcto. Ya no discutimos con el modelo: discutimos con el buscador.

sin fuente

respuesta fluida

nada contra qué contrastarla

detectar = indirecto

con fuente

respuesta + pasaje recuperado

chequeo directo entre ambos

detectar = comparar

todo el problema → entregarle el hecho correcto

La pregunta madre. Una vez que le entregamos un hecho al modelo, todo se corre a entregarle el hecho correcto. Cómo se corta, se indexa, se busca y se ordena es esa única pregunta.

Dónde estamos parados

El campo tiene un mapa. El survey de Gao et al. (2023) ordena los sistemas de RAG en una escalera de sofisticación creciente. Es el vocabulario estándar y lo usamos para saber qué vamos a ver y qué no.

Los dos escalones de hoy. Naive RAG: cortar, indexar, recuperar por similitud, generar — el pipeline mínimo que vamos a construir y romper en la primera mitad. Advanced RAG: las intervenciones antes y después de recuperar que arreglan cada rotura — la segunda mitad.

Los dos que nombramos y no desarrollamos. Modular RAG: recuperar varias veces, decidir si hace falta, reescribir la consulta — vuelve en las próximas clases. Agentic RAG: el agente decide solo cuándo y qué recuperar — es la unidad de agentes.



Gao et al. 2023 — la escalera del campo

Un mapa sirve por lo que incluye y por lo que deja afuera. Hoy subimos dos escalones. Los otros dos existen, tienen nombre y tienen fecha — no son un agujero que estemos salteando.

El loop mínimo, rearmado

Ya está construido. Cuando vimos embeddings, sobre el final, armamos un RAG completo en pocas líneas: embeber → recuperar por coseno (top-k) → generar sobre lo recuperado. Corría, con modelos locales y libres.

Las tres piezas. Un embedding de oración convierte un texto en un vector que ubica los significados parecidos cerca. La similitud coseno mide el ángulo entre dos vectores. Top-k es quedarse con los k documentos de mayor coseno contra la pregunta.

La pieza que se olvida. El generador nunca ve vectores: recibe el texto de lo recuperado, pegado en su prompt. Los vectores son sólo el mecanismo de búsqueda — por eso se puede cambiar el modelo que escribe sin re-embeber nada.



Eso ya es RAG. Recuperar y generar sobre lo recuperado. Lo de hoy no es reemplazarlo: es descubrir dónde se rompe. ▸ Notebook §1 — el baseline de embeddings, tal cual.

Lo que quedaba pendiente

La lámina con la que cerramos embeddings. Decía que para un RAG en serio faltaban tres cosas: chunking (partir documentos largos en pedazos indexables), un vector store de verdad, y reranking (un segundo paso que afina el top-k). Las tres están en el programa, aunque no seguidas: las dos primeras abren la clase, el reranking es la última pieza que montamos.

Y el callout que la acompañaba. "De este esqueleto a un RAG de producción, lo que falta es escala, no concepto." Hoy hay que corregirlo: es la mitad de la verdad y la mitad equivocada.

La corrección. También falta concepto. Cortar un documento es una decisión semántica, no de volumen: un corte malo destruye información aunque el corpus tenga diez archivos. Y una técnica de la segunda mitad no tiene nada que ver con escala: cambia qué texto se usa para buscar.

embeddings decía que faltaba

- chunking
- vector store de verdad
- reranking

y su conclusión:

falta: escala

hoy lo corregimos

falta: escala y concepto

un corte malo rompe un corpus de diez archivos

y una pieza no escala nada: cambia el texto que se busca

Hoy cobramos una promesa, no descubrimos un problema. Ya sabíamos que faltaba esto. Lo que no sabíamos es que ninguna de las tres se arregla sólo con más máquina.

Y cambiamos de embedder

El corpus de hoy está en español. El de embeddings eran cinco oraciones para la demo; hoy trabajamos sobre documentos reales en castellano. El embedder tiene que hablar el idioma de los datos, así que cambiamos a uno multilingüe.

Y por eso re-indexamos todo. Es la invariante que vimos en embeddings cobrándose: un embedder, un espacio. Los vectores de un modelo no son comparables con los de otro —a veces ni la dimensión coincide—, así que cambiar de embedder obliga a re-embeber el corpus entero. Es un costo a presupuestar.

El protocolo depende del modelo. Algunos embedders piden anteponer una marca al documento y a la consulta (search_document: / search_query:); el de embeddings las exige y el notebook no se las daba. El de hoy declara que no lleva ninguna.

La regla universal. Corpus y consulta se embeben con el mismo modelo y el mismo régimen. Lo demás lo decide cada model card.



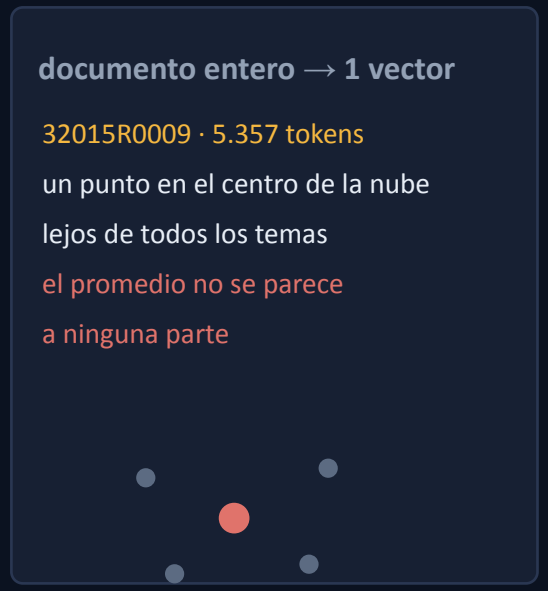
Cambiar de embedder no es cambiar una línea. Es re-embeber todo. Y antes de embeber, leer la model card: el protocolo no se adivina.

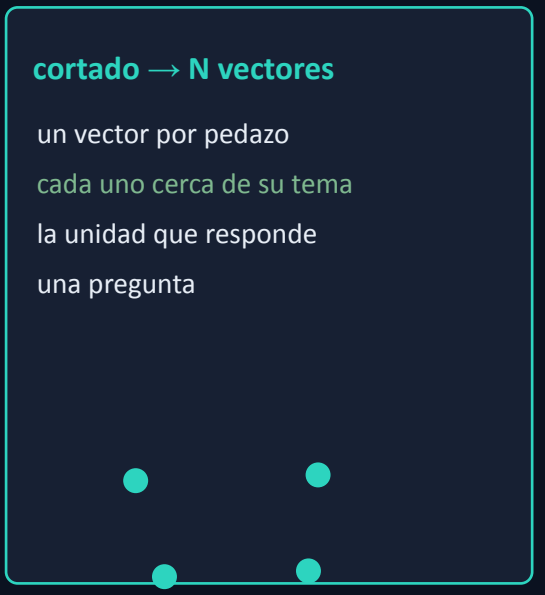
El documento no entra

El corpus real no son cinco oraciones. Son documentos de miles de palabras. El baseline de embeddings embebía cada "documento" entero porque cada documento era una frase; con archivos de verdad, ese paso deja de funcionar por dos razones.

La razón dura. El embedder tiene una ventana máxima, y lo que la excede no se embebe: se descarta. Un acto del corpus son 5.357 tokens — y cabe en los 8.192 que declara el modelo, pero no en los 1.024 que le fijamos nosotros, ni en los 512 del re-puntuador. La ventana efectiva es una decisión, no un dato del modelo.

La razón interesante. Aunque entrara, un solo vector para un documento entero es un promedio de todo lo que dice. Un reglamento que habla de diez temas queda representado por un punto que no está cerca de ninguno. Se busca "plazo de entrega" y el documento que lo contiene no aparece.





Un documento no es una unidad de recuperación. La unidad es el pedazo que responde una pregunta. Hay que cortar — y cómo se corta es una decisión, no un detalle.

El tamaño se mide en tokens, no en caracteres

La unidad es el token. Cuando vimos tokenización ya nombramos este caso: contar tokens sirve para estimar costo, verificar la ventana y partir en chunks para RAG. Un límite en caracteres o palabras se rompe distinto en cada idioma y con cada tokenizer.

El corte ciego. Partir cada N tokens es lo más simple y corta en cualquier lado. La oración que responde queda partida al medio, mitad en un pedazo y mitad en el siguiente, y ninguno la contiene entera. El coseno no encuentra ninguno.

Por qué igual se empieza por ahí. Porque es determinista, barato y predecible, y porque los dos arreglos que siguen —solapar y respetar la estructura— se entienden como reacciones a esta falla concreta.



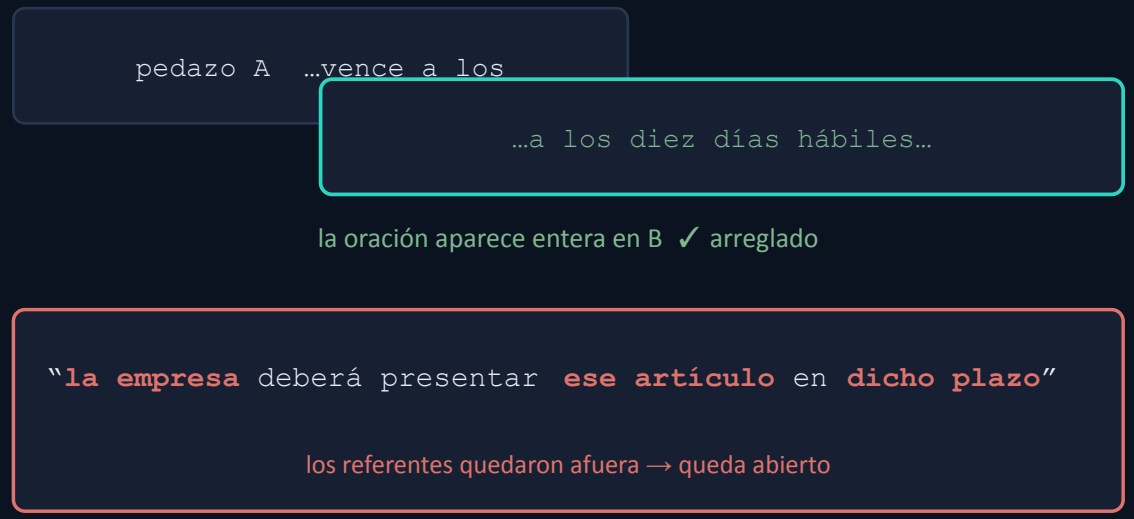
El primer parámetro de un RAG no es el modelo: es dónde se corta. El corte ciego parte exactamente lo que se iba a buscar. ▸ Notebook §4 — dos tamaños de chunk, con el acierto de recuperación al lado.

Solapar: el parche barato

El arreglo obvio. Que cada pedazo repita el final del anterior. Con un solapamiento del 10 al 20%, la oración partida aparece entera en al menos uno de los dos, y el problema anterior desaparece. Cuesta espacio duplicado y nada de complejidad.

Lo que sí arregla. La discontinuidad en el borde del corte. Es la mejora con mejor relación costo/beneficio del pipeline, y por eso es el default de casi todas las librerías.

Y el segundo daño, que no arregla. El pedazo cortado perdió sus referentes. Dice “la empresa deberá presentarlo”, “ese artículo”, “el trimestre siguiente” — y quién, cuál y qué quedaron del otro lado. Es correcto y es inútil: no se parece a ninguna pregunta que un humano haría. El solapamiento tapa el corte, no la orfandad.



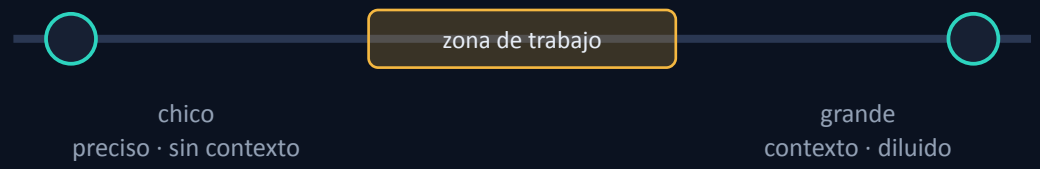
Dos daños distintos, un solo parche. Solapar arregla el corte en el medio de una oración. No arregla que el pedazo, ya cortado, no sepa de qué está hablando.

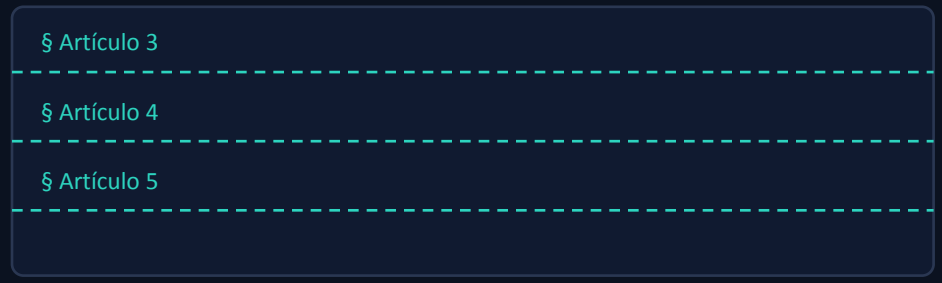
Dónde cortar, y de qué tamaño

El fix honesto: cortar donde el documento ya se corta. Los documentos reales tienen estructura —títulos, secciones, artículos—, y esas fronteras son semánticas: el autor ya decidió dónde termina una idea. Cuando el documento tiene estructura explícita, esto le gana a cualquier heurística de longitud.

El criterio de tamaño, un balance. Chico: preciso, poco ruido, pero le falta contexto y puede hacer falta varios pedazos. Grande: trae contexto, pero diluye —el vector promedia más y la señal se pierde, la falla de la lámina 8 en escala menor—. No hay número universal.

Dos variantes, nombradas. El semantic chunking corta donde cambia el tema. El sentence-window indexa oraciones sueltas y al recuperar entrega la oración con sus vecinas. Las dos son refinamientos de lo mismo: indexar una cosa y entregar otra.





cortar en las fronteras que el autor ya marcó

Cortar por estructura, dimensionar por tarea. Si el documento indica dónde termina una idea, conviene respetarlo. El tamaño no se hereda de un tutorial: se mide. ▸ Notebook §4.

El techo duro: se trunca en silencio

Falla sin error. Si un pedazo excede la ventana del modelo que lo procesa, el sobrante se descarta sin excepción, sin warning y sin log. El código corre, el índice se arma, las búsquedas devuelven resultados. Hay texto que nunca se indexó, y no hay forma de enterarse mirando la salida.

Y la ventana que aprieta no es la obvia. El reflejo es mirar la del embedder, hoy amplia: 8.192 tokens. Pero en la segunda mitad vamos a agregar un modelo que re-puntúa los resultados, y ése corta en 512. Dieciséis veces menos. El límite efectivo es el del componente más chico.

La consecuencia práctica. Los pedazos que se re-puntúan se dimensionan contra el tokenizer del modelo que los re-puntúa. Si además se entregan bloques más grandes al generador —cosa que haremos—, esos no pasan por ahí. Se re-puntúa lo chico; se entrega lo grande.



La falla más cara es la que no tira excepción. Un pipeline que corre no es un pipeline que funciona. Ésta se caza midiendo, no leyendo la consola.

Una lista en memoria no sobrevive

Lo que hacía el baseline. Los vectores de embeddings vivían en una lista de numpy, y buscar era calcular el coseno contra todos, uno por uno, y ordenar. Con cinco documentos es instantáneo. Con los miles de pedazos que acabamos de producir empieza a doler; con millones, no termina.

Y el problema no es sólo la velocidad. Esa lista vive en memoria: si se cae el proceso, se pierde. No se puede agregar un documento sin rearmar todo, ni borrar uno, ni buscar "sólo dentro de los de 2024". Un índice de producción hace esas cuatro cosas, y ninguna es velocidad.

Dos necesidades distintas. Buscar rápido entre muchos vectores, y administrar el corpus indexado. Se resuelven con herramientas distintas, y conviene no confundirlas.

lista de numpy

coseno contra TODOS

5 → 8.000 → 1.000.000

problema de algoritmo

cuatro requisitos que no cumple

persistir

agregar

borrar

filtrar

velocidad ≠ administración

Dos problemas que se parecen y no son el mismo. Buscar rápido es un problema de algoritmo. Persistir, actualizar y filtrar es un problema de base de datos.

Los tres escalones hasta el vector store

Primer escalón: la búsqueda exacta, optimizada. Una librería como FAISS (Facebook AI Similarity Search) con un índice plano calcula el mismo coseno contra todos —es el buscar() de embeddings— pero en código vectorizado. Ojo: la lista de numpy también es un índice plano. Lo que agrega este escalón no es dejar de ser plano: es estar optimizado.

Segundo escalón: dejar de mirar todos. Cuando ni eso alcanza, un índice ANN (approximate nearest neighbor) navega una estructura y mira sólo una fracción. El más usado es HNSW (hierarchical navigable small world). La palabra clave es aproximado: cambia exactitud por velocidad, y puede no devolver el mejor resultado.

Tercer escalón: administrar. Un vector store como Chroma persiste en disco, deja agregar y borrar por documento, y filtra por metadatos antes de buscar. Por dentro corre HNSW. Es lo que vamos a usar, no por rápido, sino porque el corpus hay que administrarlo.



HNSW es el motor del store, no una alternativa

No se elige un vector store por velocidad. A la escala de un proyecto de la materia, la búsqueda exacta alcanza. Se elige por lo otro: persistir, actualizar y filtrar. ▸ Notebook §5.

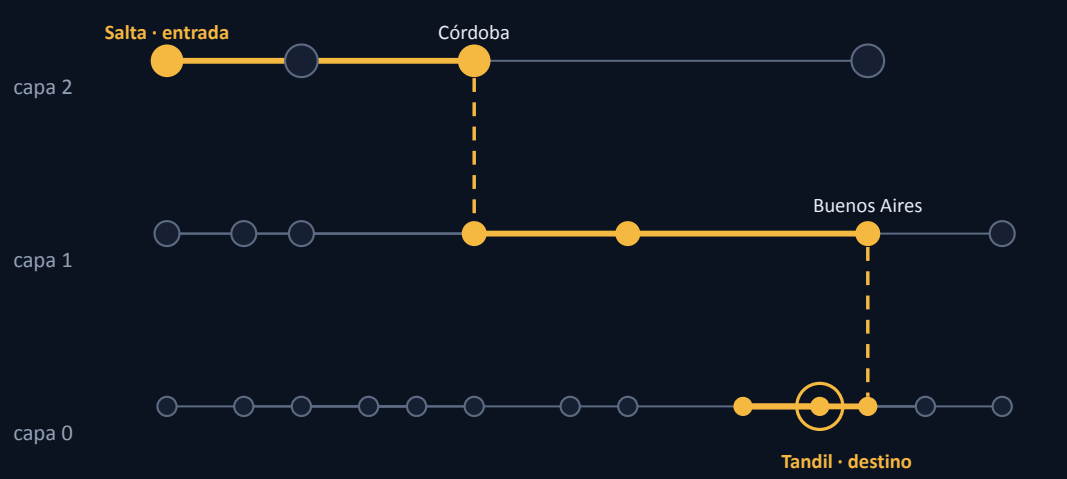
Cómo hace HNSW para no mirar todo

La idea, en una frase. Se arma un grafo donde cada vector se conecta a los que más se le parecen, y se busca caminando: parado en un nodo se miran sólo sus vecinos, se salta al más cercano a la consulta y se repite hasta que ninguno mejore. Nunca se miran los millones: se mira un camino — como un mapa donde cada ciudad conoce sólo a sus vecinas.

Por qué hay capas. Caminar de vecina en vecina para cruzar el país cuesta cientos de saltos. Por eso se apilan grafos sobre los mismos puntos: arriba, pocos nodos con aristas larguísimas para cruzar de pocos saltos; abajo, todos con aristas cortas para afinar. Es una skip list: atajos arriba, precisión abajo — avión, después ruta, después las calles del pueblo.

El recorrido, concreto. Buscando el más cercano a Tandil: en la capa alta, Salta→Córdoba; en la media, →Rosario→Buenos Aires; en la base, →Azul→Tandil. Quince nodos mirados, en vez de dos mil.

Qué se paga. Construir el índice es caro —cada inserción es en sí una búsqueda— y ocupa memoria. Y es aproximado: el descenso puede trabarse en un mínimo local y devolver el quinto en vez del cuarto, sin avisar. La perilla ef —cuántos caminos se exploran en paralelo— compra precisión a costa de latencia.



6 nodos visitados · 30.730 pedazos en el corpus

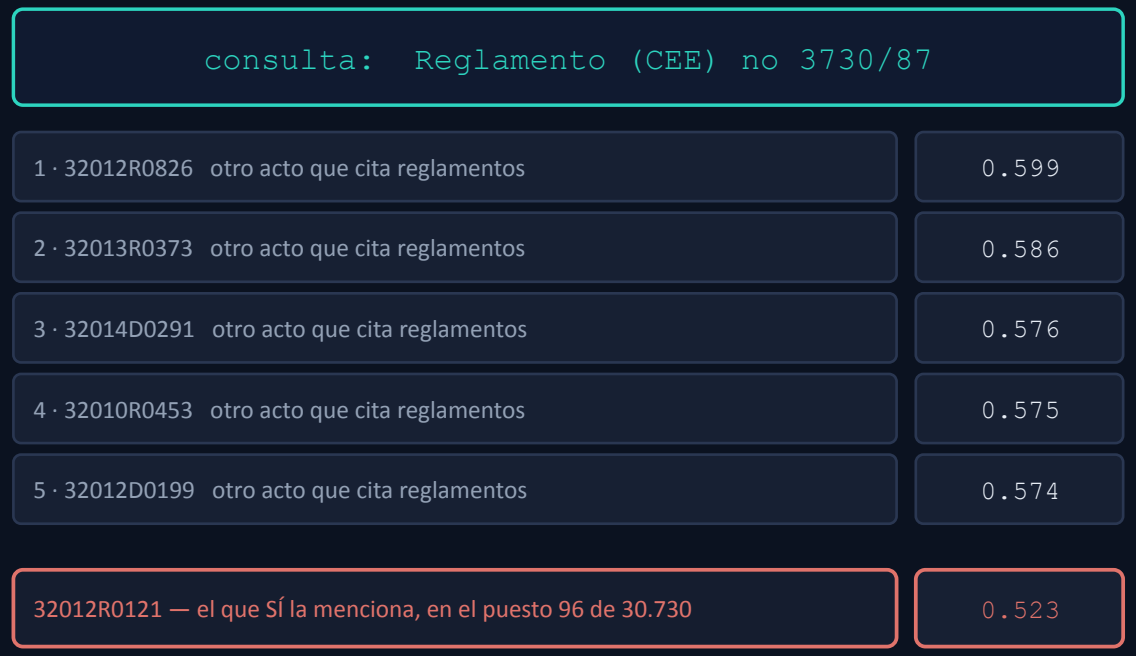
Aproximado es una decisión, no un defecto. A escala grande la pregunta deja de ser “¿cuál es el más cercano?” y pasa a ser “¿cuánta exactitud pago por la latencia?”.

La consulta que el coseno no encuentra

Una pregunta que cualquiera haría. El usuario busca un documento por su identificador, tal como aparece citado adentro de otros: Reglamento (CEE) no 3730/87. Es la búsqueda más común en un corpus normativo, y la más fácil: el string está literal en el texto.

Y el recuperador falla. El top-5 vuelve con cinco pasajes que hablan del mismo tema, con el mismo vocabulario — y el acto buscado no está entre ellos. No sale cuarto: no aparece. No devolvió nada malo; devolvió cosas parecidas, que es peor, porque el generador va a responder con ellas con total seguridad.

Lo contraintuitivo. Esto no se arregla con más documentos, ni con un embedder mejor, ni con un corpus más chico. Con diez documentos falla igual. Es la primera falla del día que no es de escala.



Lo peor que puede hacer un recuperador no es no devolver nada. Es devolver cinco cosas parecidas y confiadas, que el generador va a usar como si fueran la respuesta.

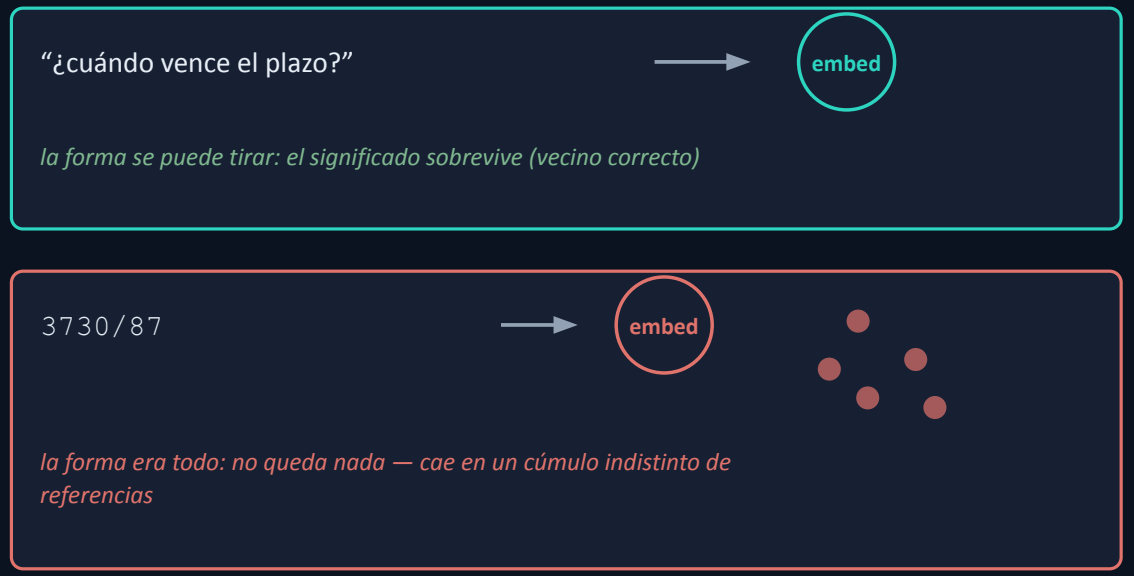
Por qué falla: es lossy por diseño

El embedding descarta la forma superficial a propósito. Queremos que “¿cuándo vence el plazo?” y “fecha límite de presentación” caigan cerca aunque no compartan palabra. Esa capacidad se consigue tirando lo que distingue una cadena de otra y quedándose con el significado.

Y un identificador es pura forma superficial. 3730/87 no significa nada: no tiene sinónimos ni campo semántico. Es una etiqueta arbitraria. El modelo lo empuja a la región de “referencias normativas en general”, donde conviven todas las citas, indistinguibles.

Lo mismo vale para toda etiqueta. Códigos de error, SKUs, números de expediente, nombres propios raros, versiones. En cualquier corpus técnico eso es un porcentaje alto de las búsquedas reales — y es donde la recuperación densa es más débil.

Queda abierta. No la arreglamos ahora: el arreglo es una pieza de la segunda mitad.



La virtud y el defecto son la misma propiedad. El embedding encuentra sinónimos porque tira la forma. Por eso mismo no encuentra un código exacto. No es un bug del modelo: es el modelo.

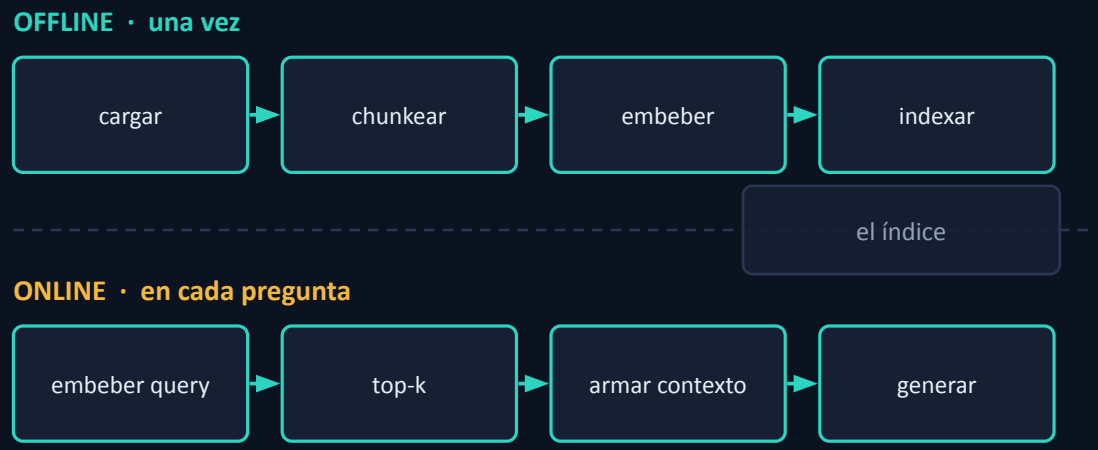
Las dos fases

El mapa que nos ganamos. Con las tres fallas ya nombradas —el documento no entraba, el índice no escalaba, el identificador no aparecía—, el pipeline completo ya no hay que postularlo: se arma solo con lo que fuimos agregando. Y se parte en dos fases que conviene nombrar.

Offline: se corre una vez, antes de atender a nadie. cargar → chunkear → embeber → indexar. Es cara, tarda, y hay que rehacerla cada vez que cambia el corpus o el embedder. Todo lo que se mueva acá es tiempo que no se paga por consulta.

Online: se corre en cada pregunta, con el usuario esperando. embeber la consulta → recuperar top-k → armar el contexto → generar. Acá cada milisegundo cuenta y cada llamada a un modelo se nota.

Por qué importa. Casi todas las técnicas de la segunda mitad se clasifican por dónde intervienen, y eso decide su costo. Offline es gratis; online se paga en cada consulta, para siempre.



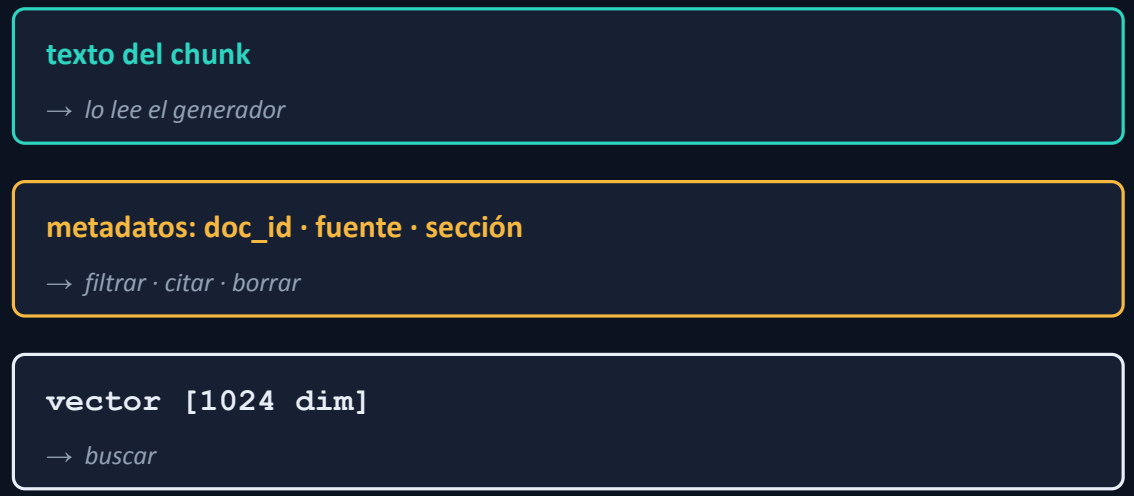
Dónde interviene una técnica decide cuánto cuesta. Offline se paga una vez; online, en cada consulta. Es el primer criterio para elegir entre dos arreglos que hacen lo mismo.

Qué guarda el índice

No guarda sólo vectores. Cada entrada del índice es un registro con tres partes: el texto del pedazo, sus metadatos —de qué documento, de qué sección, la fuente— y el vector. Suena obvio y es la pieza de la que cuelga media clase.

El texto, porque el generador lee texto. El vector sirve para encontrar la entrada; lo que se le pasa al modelo es el texto guardado ahí. Si no se guarda, hay que ir al documento original en cada consulta.

Los metadatos, porque habilitan tres cosas. Filtrar antes de buscar (“sólo documentos de 2024”), citar la fuente, y borrar o actualizar un documento sin rearmar el índice. Las tres las vamos a usar.



el vector encuentra · el texto responde · los metadatos administran

El identificador del documento es la pieza más barata y la más reutilizada. Con él se cita la fuente, se filtra, se borra — y se arma la vara con la que vamos a medir todo lo de la segunda mitad.

Una segunda copia, y un control de acceso

Indexar crea una segunda copia de los datos. Como el store guarda el texto de cada pedazo, al terminar la fase offline se tiene el corpus dos veces. Hay que versionarla, respaldarla y —sobre todo— borrarla cuando corresponde. Eliminar del origen no alcanza: siguen en el índice, y el sistema los va a seguir devolviendo.

El filtro por metadatos es el control de acceso. Con varios usuarios sobre un índice, un top-k sin filtro le devuelve cualquier pasaje a cualquiera: la búsqueda no sabe de permisos, sólo de distancias. O se parte por colección, o se filtra por usuario antes, o se chequean permisos después. No existe la opción de no decidirlo.

Por qué va acá y no en seguridad. Porque es una decisión de arquitectura del índice, y se toma en la fase offline. Después es tarde: ya se indexó todo sin el metadato que hacía falta para filtrar.

corpus original
+
índice (2ª copia)

dos copias, dos responsabilidades

3 usuarios, 1 índice
consulta sin filtro →
pasaje ajeno

la búsqueda no sabe de permisos

las tres salidas: **partir por colección** · **filtrar antes** · **chequear después**

Un índice es una base de datos con los documentos adentro. Todo lo que aplica a una base de datos —copias, borrado, permisos— aplica acá, y se decide antes de indexar, no después.

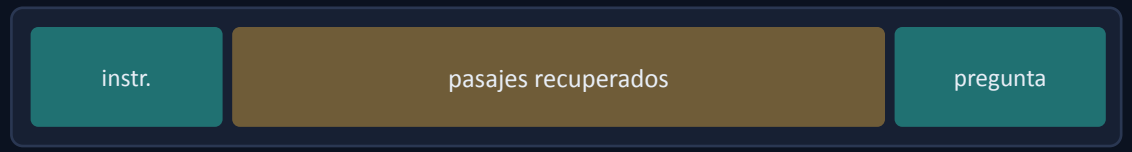
k es un presupuesto, no un parámetro libre

La pregunta que aparece sola. Si recuperar más pasajes aumenta la chance de que esté el correcto, ¿por qué no subir k y listo? Porque el contexto es un recurso finito, y en prompt engineering ya quedó sentado por qué.

Lo que ya vimos, en dos líneas. La ventana tiene un límite duro y no se usa pareja: el modelo aprovecha peor lo que cae en el medio, y la confiabilidad cae al crecer el input. Menos pasajes bien elegidos le ganan a muchos apilados.

El balance. k chico: rápido y barato, pero si el pasaje correcto no entró, ninguna técnica posterior lo salva. k grande: sube la chance de incluirlo y también la de ahogarlo en ruido, y cuesta tokens y latencia. La segunda mitad vive en esta tensión.

Y lo de la próxima clase. Qué hacer cuando la ventana igual se llena —comprimir, resumir, tirar— es el tema de la clase de memoria y gestión del contexto. Hoy decidimos qué pasaje entra; la próxima, qué hacer cuando no entra.



zona del medio: se aprovecha peor

k chico

rápido y barato

si falta el hecho, nadie lo salva

k grande

el hecho está…

…sepultado entre ruido

Recuperar no es traer todo lo que se pueda. Es gastar un presupuesto finito en los pasajes que más lo merecen. Todo lo que sigue es cómo elegirlos mejor.

El prompt que traíamos

Hasta acá miramos una sola mitad. Todo lo anterior fue sobre traer el pasaje. Pero recuperar no es responder: hay un paso más, y en el baseline de embeddings era una sola línea escrita al pasar.

El prompt, textual. “Usá SÓLO este contexto para responder.” seguido del contexto, la pregunta, y “Respondé en una sola oración.” Es lo que escribe todo el mundo la primera vez, y tiene dos agujeros.

Agujero uno: no le dimos salida. Le pedimos que use sólo el contexto, pero nunca qué hacer si no alcanza. Un modelo al que se le pide una respuesta, responde: si el pasaje correcto no está, compone algo plausible.

Y acá hay una sorpresa. Medido con el modelo de hoy, esa línea suele alcanzar: el generador dice que no sabe aunque nadie se lo pidió. Pero “suele” no es una propiedad del sistema, es una cortesía del modelo — cambia con la próxima versión, y no avisa cuándo dejó de cumplirla.

Agujero dos: los pasajes llegan anónimos. Van pegados uno atrás de otro, sin decir de dónde salió cada uno. Entonces la respuesta tampoco puede decirlo, y perdimos la trazabilidad.

```
Usá SOLO este contexto para responder.

{contexto}          # pegado, sin marcas
{pregunta}

Respondé en una sola oración.
```

← ¿y si no está? ← ¿de dónde salió cada pasaje?

Pedirlo en el prompt es una súplica, no una garantía — la misma lección de prompt engineering. Que hoy se cumpla no la convierte en garantía. Sí podemos exigir la forma de la respuesta, y verificarla.

Delimitar, abstenerse, citar

Delimitar. Cada pasaje entra marcado y separado, con su identificador de documento adelante. El modelo tiene que poder distinguir dónde empieza y termina cada uno, y de cuál salió cada afirmación. Es lo que el registro indexado nos dejó listo.

Abstenerse. La instrucción que faltaba: si el contexto no alcanza, decilo y no respondas. Suena trivial y cambia el sistema, porque convierte una falla silenciosa —una respuesta inventada— en una falla visible. Un RAG que no sabe decir "no sé" tapa sus errores de recuperación.

Citar, y citar estructurado. Además de la respuesta, se le pide el identificador del pasaje que usó. En formato libre eso se cumple a veces; con structured outputs se cumple siempre. Y da el chequeo más barato de fidelidad: ¿lo que dice está en el pasaje que dice haber usado?

La letra chica que ya vimos. El campo de razonamiento conviene antes del de la respuesta; y como las claves se serializan en orden alfabético, el orden se consigue nombrando los campos.

```
[32012R0121 · Art. 2]  <pasaje 1>   # delimitado
[32013R0373 · Art. 1]  <pasaje 2>

Si el contexto no alcanza:      # abstención
  decilo y NO respondas.

salida JSON  (orden por nombre):
  a_razonamiento: ...           # va primero
  b_respuesta:    ...
  c_cita:         doc_id        # verificable
```

Es la única pieza de hoy que toca la generación. Todo lo demás mejora qué le llega; esto mejora qué hace con lo que le llegó — y es lo que vuelve la respuesta verificable. ▸ Notebook §9.

La diferencia, medida

La prueba. Una pregunta cuya respuesta no está en el corpus —una cuota de pesca de un año posterior al último acto— contra dos prompts. El recuperador hace lo mismo en los dos casos: devuelve sus k pasajes, ninguno útil.

Sin ninguna instrucción. El modelo compone. Toma los datos que tiene delante —del año equivocado— y produce una respuesta bien escrita, específica y falsa, sin marca de duda. El contexto no evitó la alucinación, porque el contexto estaba equivocado.

Con el prompt de RAG. El modelo declara que el contexto no alcanza: basta_el_contexto en falso, cita vacía, respuesta "No lo sé". El sistema acaba de reportar su propia falla de recuperación en vez de taparla.

Y el matiz que no hay que esconder. Con la instrucción mínima de dos láminas atrás el modelo también se abstiene, casi siempre. El salto no es de "miente" a "no miente": es de una frase en prosa que hay que interpretar a un campo booleano que se puede contar y graficar. No se vuelve honesto porque se lo pidan: se vuelve auditable porque se le exige la forma.

sin instrucción

"En 2020 se podía pescar un total admisible de capturas (TAC) de bacalao…"

fluida, específica y falsa

sin marca de duda

prompt de RAG

basta_el_contexto: False

cita: []

"El contexto no contiene información sobre 2020."

un booleano, no una frase

mismo retrieval · mismo modelo · con la mínima también se abstiene, pero en prosa

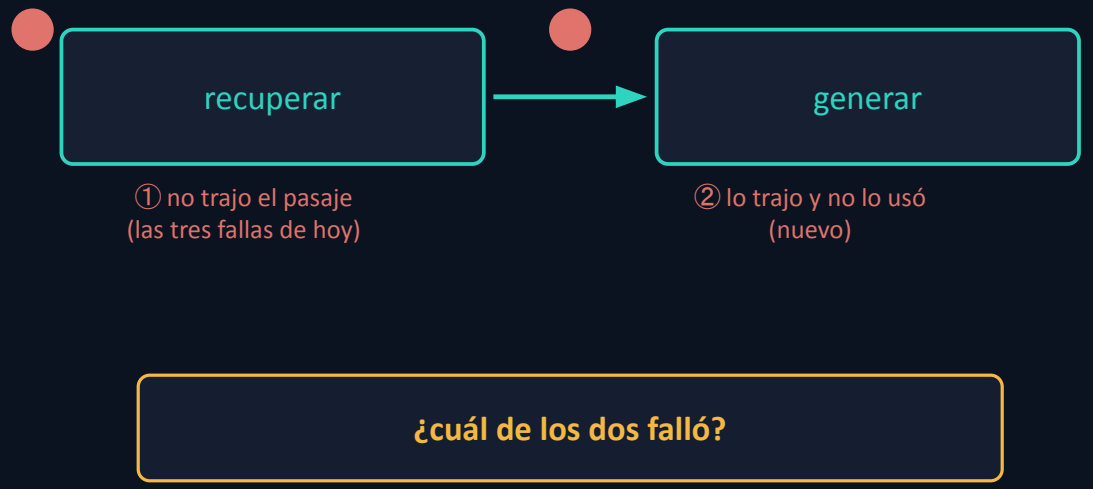
Un RAG honesto falla distinto. No falla menos: falla de una forma que se nota — y, sobre todo, de una forma que se puede contar. Eso es lo que después se puede medir.

¿Y si falla con el pasaje correcto adentro?

Las tres fallas del día fueron de lo mismo. El documento no entraba —lo arregló el chunking—, el índice no escalaba —lo arregló el vector store—, el identificador no aparecía —sigue sin arreglo—. Las tres son de recuperación: el pasaje correcto nunca llegó al modelo, y por eso la respuesta no podía ser buena.

Pero recién vimos otra cosa. En la lámina anterior, el modelo con el prompt viejo inventó teniendo pasajes delante. Cambiemos un detalle: ¿qué pasa si el pasaje correcto sí está entre los recuperados, y la respuesta igual sale mal?

Pasa, y es un modo de falla distinto. El modelo puede ignorar el pasaje, mezclarlo con lo que cree saber, o contradecirlo. Con un modelo chico local —el nuestro— no es raro. Y es importante que sea un modo aparte, porque se arregla con herramientas distintas y se mide por separado.



Un pipeline de dos etapas tiene dos formas de romperse. Y si no se separan, cada vez que una respuesta salga mal se termina adivinando cuál de las dos fue.

Las dos superficies

Dos preguntas, no una. ¿Recuperó el pasaje correcto? Y, aparte: ¿la respuesta se mantuvo fiel a lo que se le entregó? Cruzadas dan cuatro casos, y dos de ellos son los que enseñan.

“Fiel a basura”. Recuperó mal y el modelo fue perfectamente fiel a lo que recibió. Coherente, bien escrita, sostenida por el contexto — y falsa, porque el contexto era el equivocado. Justifica toda la clase: el generador no repone lo que no le llegó. La recuperación fija el techo.

“Tenía el dato y contestó otra cosa”. Recuperó bien y el modelo igual se desvió. Impide que la frase anterior sea circular: el techo es una cota superior, no una garantía. La generación decide cuánto se alcanza.

Y los otros dos. Recuperó bien y fue fiel: el caso bueno. Recuperó mal y además se desvió: el caso perdido, que no aporta información porque falló todo.



Dos superficies de falla, y hay que mirarlas por separado. Una sola métrica al final dice que salió mal, no dónde. La segunda mitad ataca la primera superficie; medir dirá si valió la pena.

Naive no quiere decir mal hecho

Lo que tenemos. Un RAG completo: corta por estructura con solapamiento, indexa en un store con metadatos, recupera por similitud, arma un contexto con pasajes identificados, y responde citando o diciendo que no sabe. Anda.

Y sabemos exactamente dónde se rompe. Y no son las tres del arranque: de aquéllas, dos ya están arregladas. De recuperación: sigue sin encontrar identificadores exactos —la única que se repite— y los pedazos pierden sus referentes al cortarse, falla que creó el arreglo anterior: cortar salvó el documento y huérfanó los pedazos. De generación: el modelo puede desviarse del pasaje aunque lo tenga.

Y una aclaración sobre la palabra. Naive es la etiqueta del campo para este pipeline, y suena peyorativa sin serlo: para la mayoría de los casos reales, esto es lo que hay que construir. Agregar piezas antes de medir un problema es hacerse daño. Lo que sigue no son mejoras que van siempre: son arreglos.

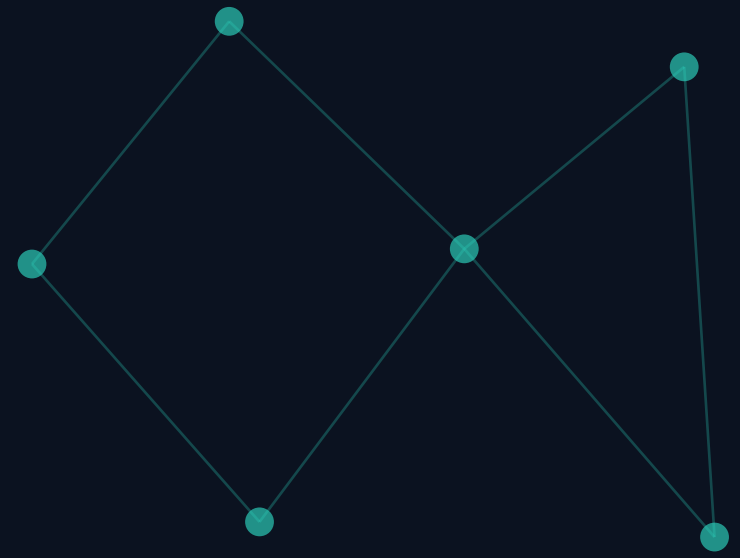


una viene del arranque · dos aparecieron en el camino — lo que sigue son arreglos

Naive no quiere decir mal hecho: quiere decir sin arreglos todavía. La segunda mitad no agrega sofisticación por sofisticación. Cada pieza que viene cuelga de una de estas tres fallas.

CLASE 8 · UNIDAD 4 · SEMANA 8

RAG avanzado



Ensanchar, afinar, y después medir:
cada arreglo cuelga de una falla que ya tiene nombre.

Volvamos a la consulta que no encontró nada

Quedó una sin arreglar. El identificador exacto: Reglamento (CEE) no 3730/87 entra, y el top-5 vuelve con cinco pasajes parecidos sin el acto correcto. Dijimos que no era un problema de escala y que no se arreglaba con un embedder mejor.

Y tenemos el diagnóstico. El embedding tira la forma superficial a propósito; un identificador es sólo forma superficial. El canal que tenemos es ciego a eso, y ningún ajuste de ese canal lo va a arreglar.

Lo que queda, entonces. Si el canal es ciego a una clase entera de consultas, la salida no es mejorar el canal: es agregar otro que vea justamente eso. Ese es el primer arreglo — pero antes hace falta algo más básico.



Antes de arreglar algo, hay que poder ver si mejoró. Vamos a agregar cuatro piezas. Sin una vara, no hay forma de saber cuál sirvió — y con qué corpus.

Recall y precision, sobre una lista concreta

El objeto. Una consulta, y los diez pedazos que el sistema devolvió. De esos diez, dos son realmente relevantes —marcados a mano por nosotros, que sabemos la respuesta—. Todo lo que sigue se mide sobre una lista así.

Recall: de los relevantes, cuántos entraron. Si el pedazo correcto no entró en la lista, se terminó: ninguna técnica posterior lo salva. No se puede re-ordenar algo que no está. Por eso las primeras tres piezas atacan esto y sólo esto.

Precision: de los que entraron, cuántos sirven. Con recall resuelto, el correcto está pero perdido entre ocho que no sirven, y el generador lee ruido. Ahí interviene la cuarta pieza.

El orden no es casual. Primero ensanchar —que el correcto esté entre los candidatos—, después afinar —que quede arriba—. Al revés no tiene sentido.



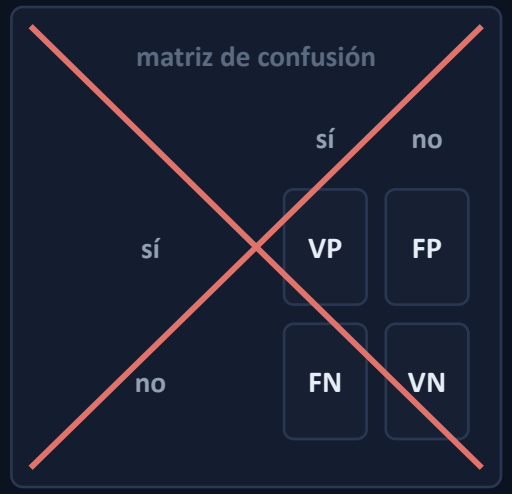
Dos problemas, en orden obligatorio. Si el pasaje correcto no entró, afinar no sirve de nada. Toda la segunda mitad son estas dos palabras: primero ensanchar, después afinar.

No es la precision de la clasificación

La advertencia. Si precision y recall ya aparecieron en clasificación, fue sobre una matriz de confusión: un clasificador que dice sí o no sobre cada caso, y se cuentan aciertos y errores. Las palabras son las mismas; el objeto no.

Las tres diferencias que importan. Acá hay una lista, no una decisión binaria por caso. Hay un orden: no es lo mismo que el correcto salga primero o séptimo. Y hay un corte en k: lo que queda del otro lado no existe para el generador.

Por qué decirlo ahora. Porque el reflejo de traducir a la matriz de confusión lleva a conclusiones equivocadas — por ejemplo, pensar que subir k "baja la precision" de forma fatal, cuando la métrica que vamos a usar tiene en cuenta la posición.



decisión sí/no por caso

la misma lista, tres cortes

k	recall	precision
5	100 %	20 %
10	100 %	10 %
50	100 %	2 %

un solo relevante en los tres

lo único que creció fue el denominador

lista · orden · corte en k

Mismas palabras, otro objeto. En recuperación hay una lista, hay un orden y hay un corte. Eso cambia qué significa mejorar.

Una vara que no necesita juez

Lo más barato que existe, y alcanza. Una tabla escrita a mano: la pregunta, y el identificador del documento que la responde. Veinte a treinta filas. Sin modelo, sin librería, sin API.

Qué se mide con eso. ¿Entró el documento correcto entre los k recuperados? —da un porcentaje—. Y ¿en qué puesto quedó? —que no necesita fórmula: si sube de la posición siete a la dos, mejoró—.

La decisión de diseño. Se etiqueta el documento, no el pedazo. Porque en dos láminas vamos a re-cortar el corpus, y etiquetas que apuntaran a pedazos quedarían inválidas. El documento sobrevive al re-chunkeo.

Por qué sin LLM. Un eval set generado por un modelo se muerde la cola: se estaría midiendo si el sistema recupera lo que un modelo cree que es la respuesta. Y escribir treinta preguntas a mano obliga a mirar el corpus propio.

```
pregunta                                    | doc_id
--------------------------------------------+-----------
¿cómo se reparte el dinero de vigilancia? | 32010D0352
¿qué mantiene a un avión en condiciones?   | 32014R1321
¿cuándo se cierra una zona de pesca?      | 32010R0724
30 filas, escritas a mano y parafraseadas
```

```
técnica    | ¿entró? | puesto
-----------+---------+--------
baseline   |    .    |   .
+ léxico   |    .    |   .
...        |    .    |   .
```

se etiqueta el documento, no el chunk

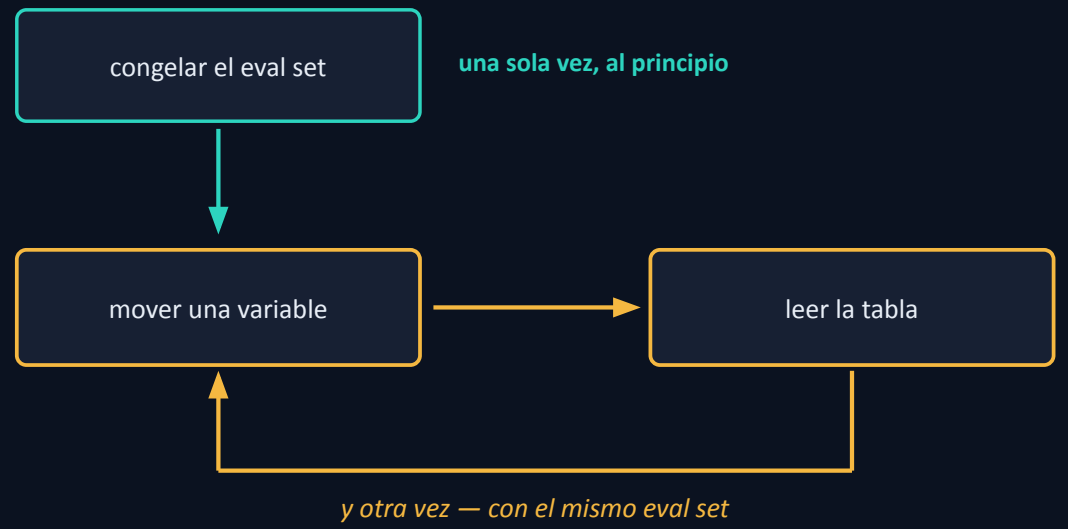
La métrica más útil del día no usa un modelo. Treinta filas y dos columnas. Cada pieza que agreguemos llega con su número en esta tabla. ▸ Notebook §10 — el eval set y la tabla de resultados.

El protocolo

Tres pasos, siempre los mismos. Se congela el eval set. Se mueve una variable. Se lee la tabla. Si se mueven dos cosas a la vez y el número sube, no se sabe cuál fue — y la próxima decisión es a ciegas.

Esto es lo que se evalúa en un proyecto. No la cantidad de técnicas apiladas: la capacidad de decir “probé esto, lo medí así, mejoró tanto, por eso lo dejé” —o “lo medí, no mejoró, por eso lo saqué”, que vale igual. Cuatro piezas sin medir es peor que dos justificadas.

La advertencia que va en la lámina. Los números salen de un eval set de treinta filas sobre el corpus de esta clase. Sirven para comparar entre sí las técnicas de hoy. No son benchmarks, no son generalizables, y una diferencia chica puede ser ruido.



medido sobre el corpus de la clase — no es un benchmark

Una variable por vez. Es la diferencia entre tener un pipeline y tener una colección de cosas que alguien dijo que funcionaban.

El canal que ve lo que el embedding pierde

Hay otra forma de buscar, y es la vieja. Antes de los embeddings, buscar era contar palabras: qué términos de la consulta aparecen en el documento, cuántas veces, y cuán raros son en el corpus. El algoritmo estándar se llama BM25 y tiene más de veinte años de uso industrial.

Matchea la palabra literal, no el significado. Y eso, que es su limitación famosa, acá es lo que necesitamos: 3730/87 es un término rarísimo, así que cuando aparece BM25 lo puntúa altísimo. No entiende nada, y por eso acierta.

¿Y cuando la consulta es una pregunta entera? Ahí no hay un término salvador: BM25 parte la oración en sus palabras de contenido y suma el aporte de cada una, con las raras pesando mucho más que las comunes.

Es el espejo exacto del otro canal. El embedding tira la forma y conserva el significado; BM25 conserva la forma y no entiende el significado. En la industria el par tiene nombre: dense retrieval el de embeddings, sparse retrieval el de BM25.

denso (embeddings)

✓ “fecha límite” ↔ “¿cuándo vence?”

✗ pierde 3730/87

conserva el significado,

tira la forma

léxico (BM25)

✓ encuentra 3730/87

✗ pierde sinónimos

conserva la forma,

ignora el significado

mismo corpus · dos consultas · dos canales ciegos a cosas distintas

No es un canal peor: es un canal ciego a otra cosa. Y lo que le falta a uno es justamente lo que le sobra al otro.

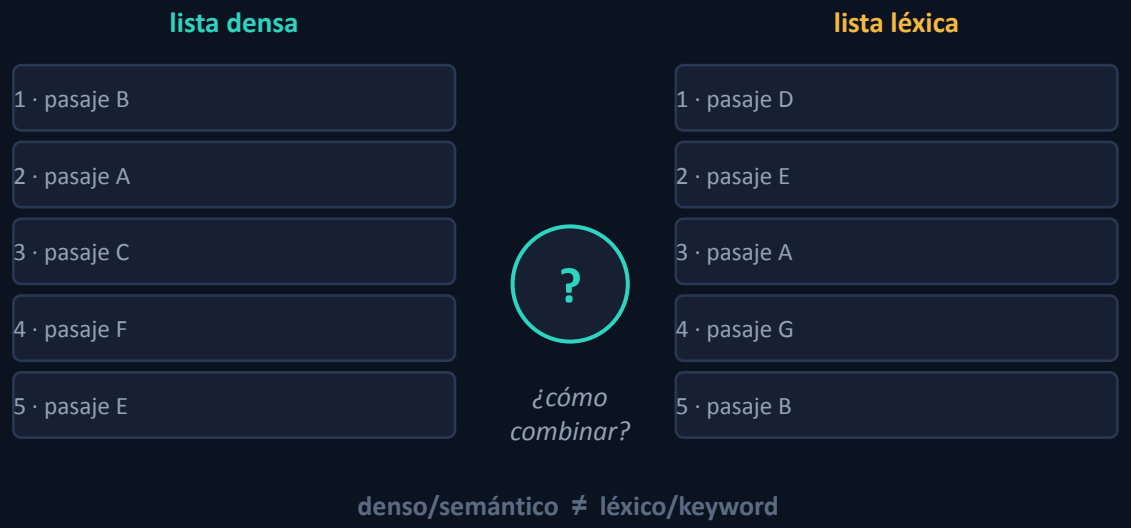
Búsqueda híbrida: el keyword vuelve, y no es un retroceso

Suena a retroceso, y no lo es. El curso enterró la búsqueda por palabras clave: fallaba con sinónimos, no entendía reformulaciones, era frágil. Todo eso sigue siendo cierto. Lo que cambió es que ahora sabemos que el coseno falla al revés.

La conclusión no es elegir. Es usar los dos. Cada canal es ciego a algo distinto, así que la unión ve más que cualquiera por separado. Esto se llama búsqueda híbrida, y es el default en producción, no una optimización exótica.

Una precisión de vocabulario. "Semántico" y "denso" son el canal de embeddings. "Léxico" o "keyword" es el de coincidencia exacta. BM25 no es semántico, aunque a veces se lo meta en la misma bolsa.

Lo que falta resolver. Tenemos dos listas de resultados, cada una con sus puntajes. Hay que combinarlas en una sola — y ahí aparece un problema que no es obvio.



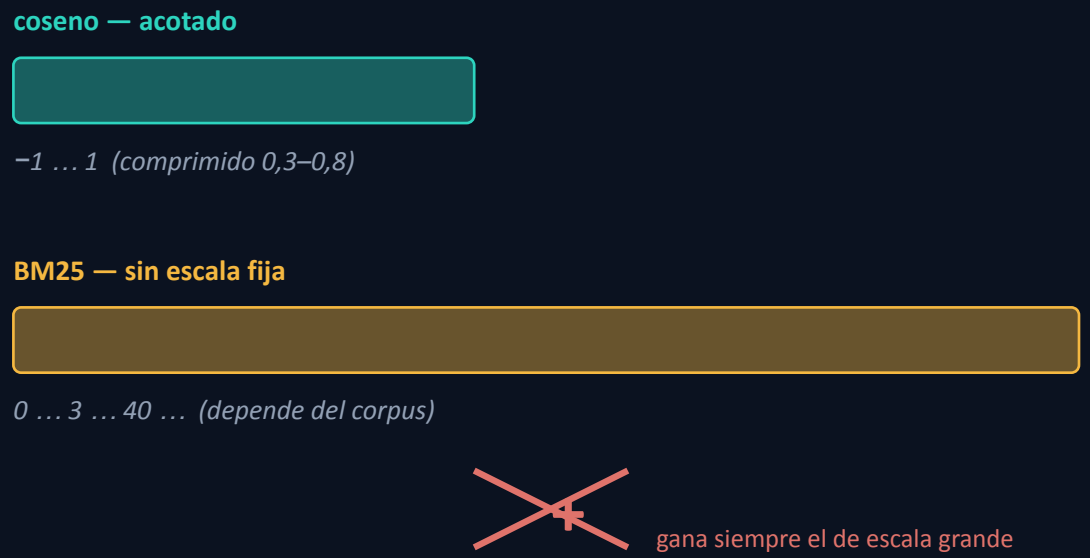
Dos canales ciegos a cosas distintas ven más que uno bueno. Por eso híbrido es el default, y no un lujo.

Los puntajes no se pueden sumar

El reflejo: sumar. Cada documento tiene un puntaje en cada lista, se suman y se ordena. Parece razonable y está mal, por una razón que conviene entender antes de ver el arreglo.

Las escalas no tienen nada que ver. El coseno está acotado: va de −1 a 1, y en la práctica los relevantes se apiñan entre 0,3 y 0,8. BM25 no tiene cota ni escala fija: depende de cuántos términos raros coincidieron, de la longitud y de la estadística del corpus. Puede valer 3 o 40.

Qué pasa si igual se suman. El canal de escala más grande domina. El otro deja de influir, y queda un híbrido que en los hechos usa un solo canal — con el costo de los dos. Normalizar tampoco alcanza: los puntajes no son comparables ni entre dos consultas del mismo canal.



Antes de fusionar, conviene preguntarse si lo que se está sumando es la misma unidad. Acá no lo es — y esa es la razón de existir de lo que viene.

Tirá el puntaje, usá el puesto

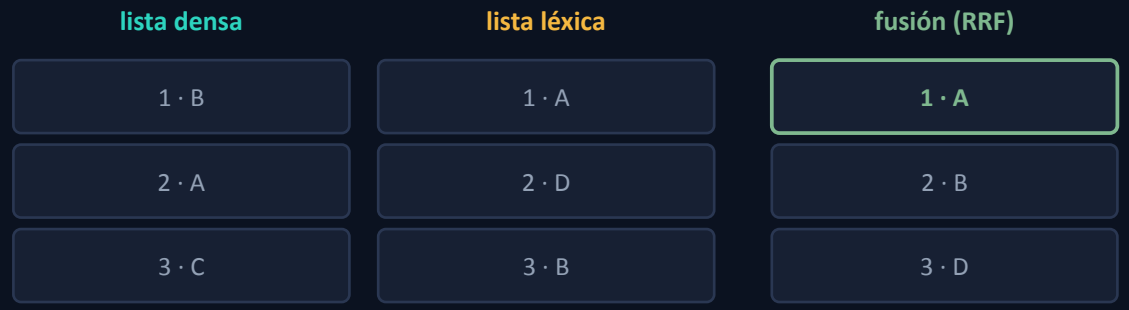
La idea, y es elegante. Si los puntajes no son comparables, no se usan. Lo que sí es comparable entre dos listas cualesquiera es el puesto: salir primero significa lo mismo en las dos. Se llama reciprocal rank fusion, RRF.

Cómo se calcula. Cada documento suma, por cada lista en la que aparece, 1/(60+puesto). Se ordena por ese total. Primero en las dos: suma dos veces. Primero en una y ausente en la otra: una sola vez.

Lo que premia: el consenso. Un documento tercero en las dos listas le gana a uno primero en una sola: dos canales independientes coinciden, y esa coincidencia es más informativa.

Y cierra la falla abierta. La consulta con el identificador: el denso sigue sin encontrarlo, el léxico lo pone primero, y la lista fusionada lo trae. No hubo que tocar el embedder.

El supuesto que nadie enuncia. Supone que los dos canales fallan de forma independiente. Con una cita pelada no es cierto: el denso va al cúmulo de referencias y BM25 a los mismos tokens genéricos. Ahí el acuerdo no es evidencia, es error correlacionado — y la fusión lo amplifica.



$$1/(60+3) + 1/(60+3) \;>\; 1/(60+1)$$

identificador 2/15 → 6/15 · sin tocar el embedder

El puesto sobrevive a la incomparabilidad de los puntajes. Por eso RRF funciona sin normalizar nada, y por eso lo escribimos a mano. ▸ Notebook §7 — BM25 y RRF, a mano.

Qué cuenta BM25 exactamente

Tres factores multiplicándose. Cuántas veces aparece el término en el documento; cuán raro es ese término en el corpus —uno que está en todos no distingue nada—; y una corrección por longitud, para que un documento largo no gane sólo por tener más palabras.

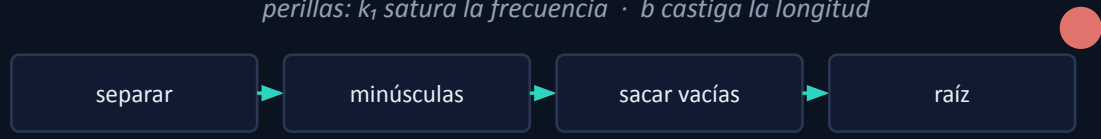
Las dos perillas. Una satura la frecuencia: la décima aparición aporta mucho menos que la segunda. La otra controla cuánto castiga la longitud. Los valores por defecto funcionan bien y casi nadie los toca.

Para usarlo en castellano. El algoritmo cuenta términos, así que hay que decidir qué es un término: separar, pasar a minúsculas, sacar palabras vacías, y reducir variantes a una raíz —si no, “plazo” y “plazos” son distintos—. Ese preproceso se aplica idéntico al corpus y a la consulta.

Una trampa concreta. Si el preproceso parte los identificadores por la barra, 3730/87 deja de existir: quedan 3730 y 87, dos números que aparecen en cualquier acto. Se cae justo la búsqueda que este canal vino a arreglar.

puntaje = Σ frecuencia × rareza × longitud

perillas: k_1 satura la frecuencia · b castiga la longitud



parte 3730/87 → 3730 + 87

```
¿Qué gravamen pagan los radiadores de aluminio desde China?

  gravamen   ✗   el acto dice “derecho antidumping”
  radiador   ✓   rarísimo en el corpus      → mucho
  aluminio   ✓   rarísimo                   → mucho
  China      ✓   frecuente                  → poco
```

acierta por dos palabras de seis — y porque la pregunta conservó «radiadores de aluminio»

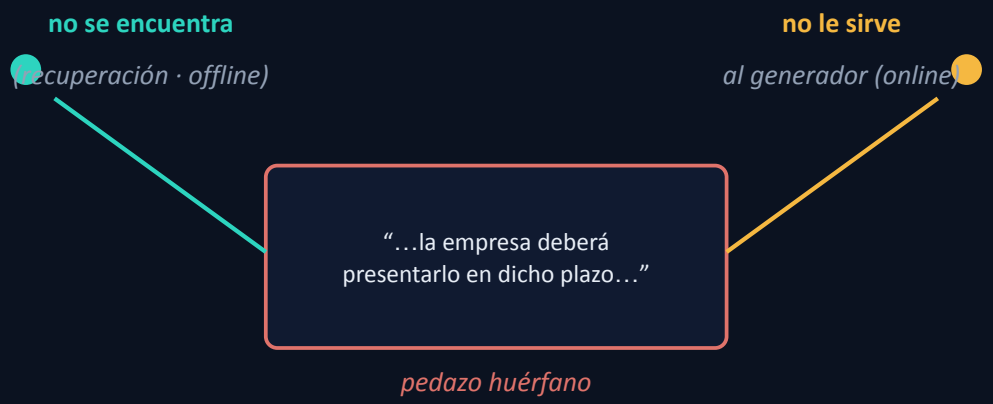
El algoritmo es viejo y estable; lo que se rompe es el preprocesamiento. Y se rompe igual para el corpus y para la consulta, o no se rompe nada.

El pedazo que perdió sus referentes

Volvemos a la que dejamos anotada. Al cortar, el pedazo quedó lleno de “la empresa”, “ese artículo”, “dicho plazo” — y los referentes quedaron del otro lado del corte. Dijimos que el solapamiento no lo arreglaba y lo dejamos para acá.

Tiene dos mitades. Recuperación: el pedazo se indexó mutilado, así que su vector representa un texto que no nombra las cosas de las que habla; una consulta que sí las nombra no se le parece. Entrega: aunque lo trajeras, al generador le llega un fragmento sin contexto.

Por qué importa la separación. Son dos problemas distintos, con dos arreglos distintos, en dos fases distintas. Uno se arregla antes de indexar; el otro, al momento de entregar. Y hacen falta los dos: arreglar sólo el segundo deja el pedazo tan invisible como estaba.



una falla, dos mitades — offline y online

Una falla, dos mitades. Se arreglan por separado, y no son intercambiables: si sólo se arregla la entrega, el pedazo sigue sin aparecer.

ENSANCHAR · CONTEXTUAL RETRIEVAL · ANTES DE EMBEBER

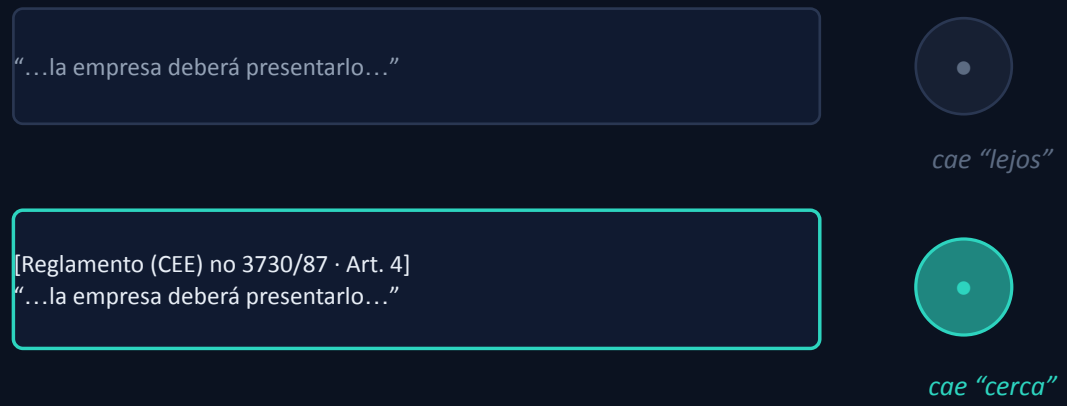
Arreglar lo que se indexa

El arreglo más directo, y es offline. Antes de embeber cada pedazo, se le prepende el contexto que perdió: el título del documento y el encabezado de la sección. El pedazo que decía “la empresa deberá presentarlo” pasa a decir de qué documento y de qué artículo viene. La técnica se llama contextual retrieval.

Qué cambia, exactamente. No se toca el embedder, ni el generador, ni la búsqueda. Cambia el texto que se indexa — y con eso, dónde cae el vector.

Lo barato y lo caro. La versión determinística —pegar título y encabezado, que ya están en los metadatos— cuesta nada. La ambiciosa, que popularizó Anthropic, hace que un modelo escriba una glosa por pedazo: una llamada por pedazo.

El caveat, y el resultado. La baja de ~⅓ en fallas es de esa variante —la que genera el contexto con un modelo—, no de la nuestra; sobre el atajo determinístico el mismo trabajo reporta “very limited gains”. Y lo medimos: el temático pasó de 26/30 a 23/30 y el identificador no se movió. Tres sobre treinta no alcanzan para llamarlo caída — pero de la mejora no hay rastro.



gratis · en nuestro corpus no movió la aguja · el −⅓ es de la variante con modelo

Para esto sirve la vara. La técnica es real y la fuente es seria; en este corpus, con la variante barata y treinta preguntas, no se mueve nada. Sin medirlo la habríamos dejado puesta, pagando el re-indexado.

Arreglar lo que se entrega

La otra mitad, y es online. Se indexan pedazos chicos, precisos para buscar. Pero cuando uno gana, no se le entrega ese al generador: se le entrega el bloque más grande que lo contiene — el artículo entero. Se la encuentra como parent-document retrieval y también como small-to-big.

Por qué funciona. Son dos requisitos opuestos. Buscar quiere pedazos chicos: un vector por idea, sin dilución. Responder quiere pedazos grandes: el contexto completo alrededor del dato. Al separar qué se indexa de qué se entrega, cada uno se optimiza por su lado.

La implementación es trivial. Un diccionario de hijo a padre, y un conjunto para no entregar el mismo padre dos veces. Diez líneas. Lo difícil no es el código: es haber visto que eran dos problemas.

Se conecta con el truncado. Se busca y se re-puntúa sobre los hijos —que caben en la ventana chica del re-puntuador—; lo que se entrega son los padres, que nunca pasan por ahí.



se indexa el hijo · se entrega el padre · los padres no pasan por el re-puntuador

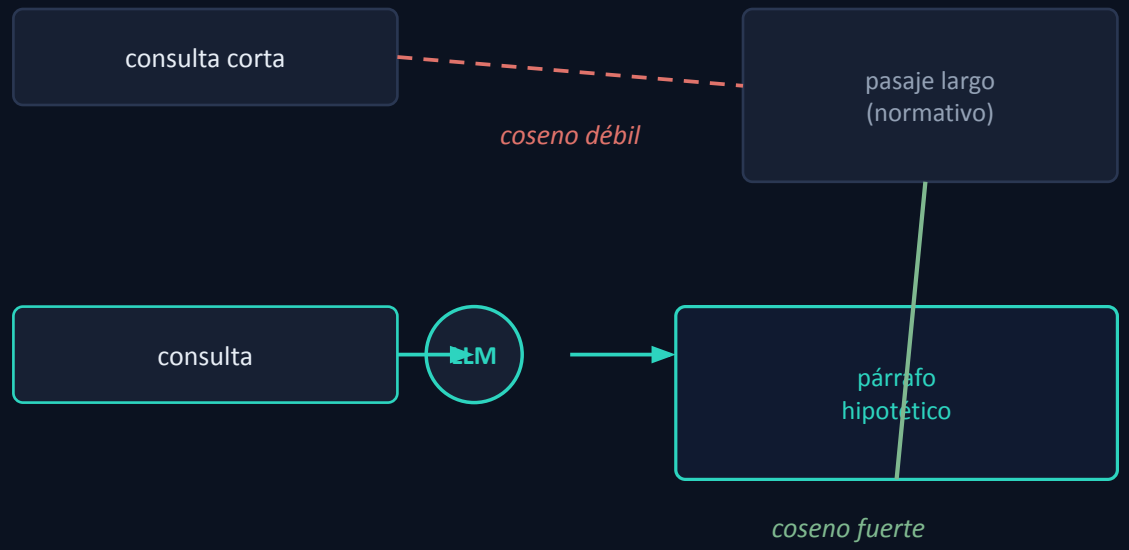
Lo que se indexa y lo que se entrega no tienen por qué ser lo mismo. Es la idea más reutilizable del día — y hace falta junto con la anterior, no en lugar de ella. ▸ Notebook §8 — el mapa hijo→padre.

La consulta no se parece al pasaje

Un problema que ya tiene nombre. Al ver embeddings lo llamamos recuperación asimétrica: una consulta corta busca un pasaje largo que la responde. No son textos del mismo tipo, y no están escritos igual.

Por qué complica el coseno. "¿Cuánto tiempo tengo para el descargo?" y el párrafo que lo responde —en registro normativo, otro vocabulario, diez veces más largo— no se parecen como textos, aunque uno responda al otro. Medimos parecido entre cosas que no tienen por qué parecerse.

La idea de HyDE, y es rara. Si el problema es que la consulta no tiene forma de pasaje, hay que darle forma de pasaje: se le pide al modelo que escriba una respuesta hipotética, y se embebe eso en vez de la pregunta. El texto con el que se busca ahora es del mismo tipo que los buscados. HyDE es un caso de una familia con nombre: transformación de la consulta, o query expansion.



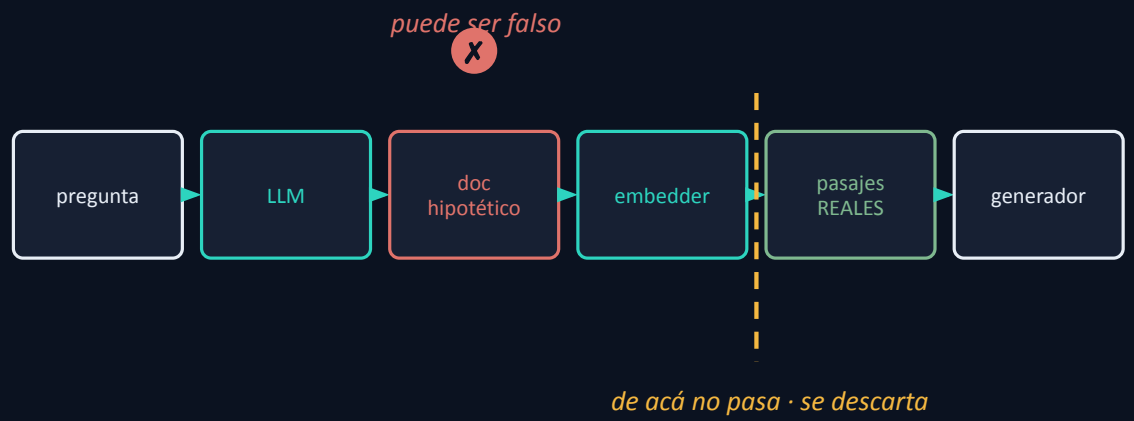
El truco no cambia el modelo ni el índice: cambia el texto con el que se busca. Es la única pieza del día que interviene sobre la consulta.

Un documento falso que funciona

La objeción obvia. Abrimos la clase diciendo que el modelo inventa y que por eso necesitábamos una fuente. Veinte minutos después le pedimos que invente un documento. ¿No es exactamente lo que queríamos evitar?

No, y la razón es dónde termina. El documento hipotético es una llave de búsqueda: se genera, se embebe, y se descarta. Nunca llega al generador, nunca entra en el contexto de la respuesta final. Al contexto entran los pasajes reales que esa llave ayudó a encontrar.

Por eso puede estar mal y servir. Lo que aporta no son los hechos: es el patrón de relevancia —el tema, el vocabulario, el registro, la forma de pasaje—. Puede inventar una fecha equivocada y aun así caer en la región correcta, donde están los pasajes con la fecha buena.



Se descarta antes de responder. El texto inventado nunca toca la respuesta: sólo apunta la búsqueda. Y si el modelo no conoce el dominio, apunta mal — ahí HyDE empeora la recuperación.

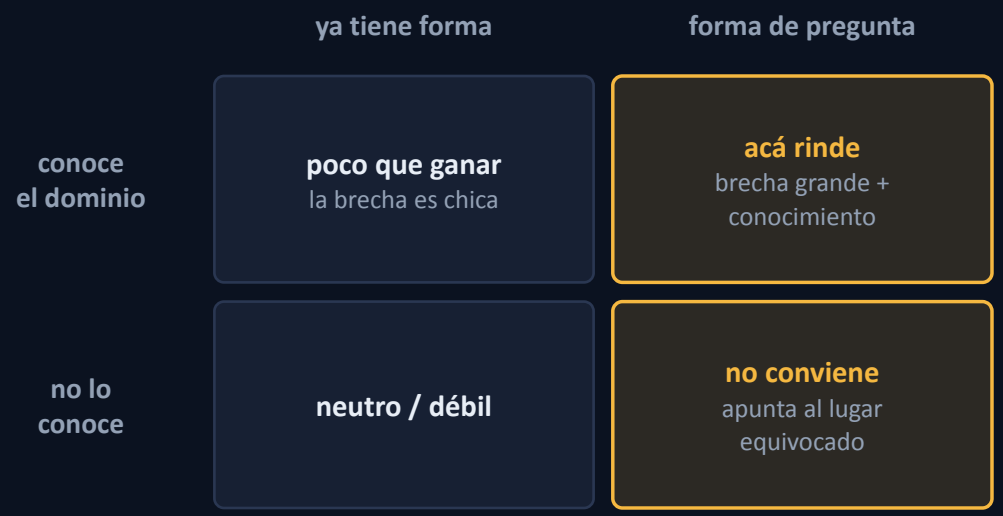
Cuándo conviene no usarlo

Depende de que el modelo sepa del dominio. Si se le pide una respuesta hipotética sobre un corpus que nunca vio, escribe un párrafo genérico que apunta al lugar equivocado. No es neutro: es peor que embeber la pregunta original, porque se agregó ruido con forma de confianza.

Cuesta una llamada por consulta. Y en la fase online, con el usuario esperando. Con un modelo local en una GPU modesta, son segundos por búsqueda. Es la pieza más cara de la segunda mitad y la única cuyo costo se paga en cada pregunta.

Dónde sí rinde. Consultas cortas y vagas sobre dominios generales, donde la brecha de forma es grande y el modelo tiene conocimiento previo. Dónde no: preguntas ya largas y específicas, o corpus muy técnicos y cerrados — el caso típico de un RAG empresarial.

Y en nuestro corpus hizo lo contrario. Fue el mayor salto del día en identificadores: 6/15 a 11/15. Pero el canal denso nunca encontró el acto correcto. Lo que hizo el párrafo hipotético fue romper un acuerdo falso entre los dos canales, que sepultaba lo que el léxico ya tenía primero.



Midió bien, y por la razón equivocada. Fue el mayor salto del día — y no por lo que HyDE promete. Sin abrir la caja lo habríamos atribuido al mecanismo del paper, y en otro corpus la mejora desaparecería sin explicación.

El vector se calculó antes de que existiera la pregunta

Todo el día ampliamos qué trae la búsqueda. Ninguna pieza tocó en qué orden lo trae. Y el orden lo decide un coseno contra un vector que se calculó offline, antes de que existiera la consulta — en un sistema real, semanas antes.

Ahí está la falla, y es estructural. Es la forma del método. Al embeber cada pasaje por separado, el modelo comprime trescientos cincuenta tokens en un vector sin saber qué le van a preguntar. Un pasaje que habla de tres cosas tiene un solo vector para las tres: si la pregunta va por la tercera, el coseno lo ordena por el promedio de las otras dos.

¿Hay otra forma de comparar? Sí, y es la más obvia: meter la consulta y el pasaje juntos al modelo, y que la red lea las dos cosas a la vez. Así comparaba textos BERT antes de que existiera SBERT.

Y la descartamos, con la cuenta hecha. Metiéndolos juntos no queda ningún vector guardable: hay que correr el modelo una vez por cada par. Diez mil oraciones entre sí son unos cincuenta millones de pasadas, sesenta y cinco horas. Ése es el motivo por el que existe el embedder que usamos todo el día.



La descartamos por cara, no por mala. Esa distinción parecía un detalle cuando vimos embeddings. Es la que abre todo lo que viene.

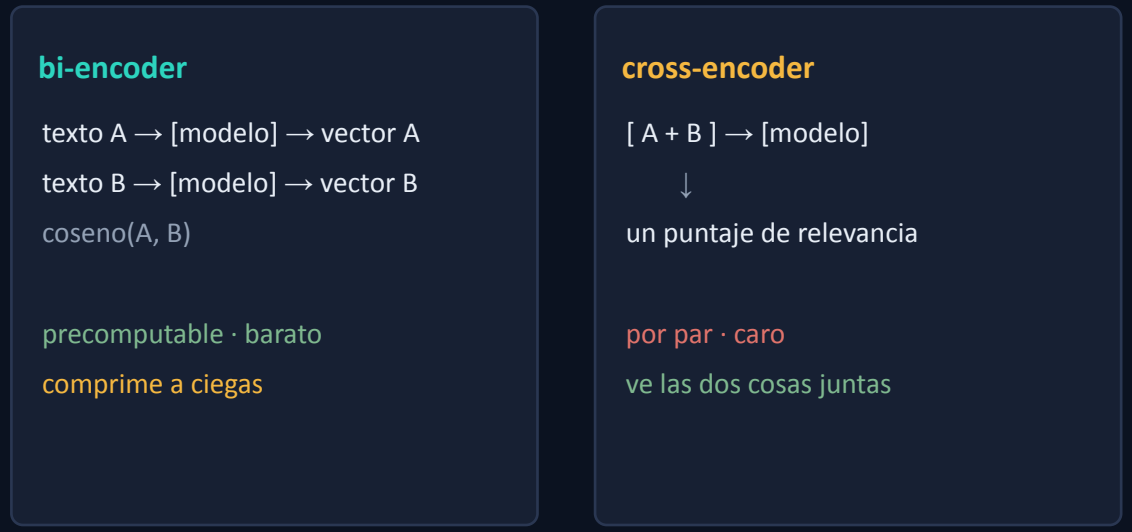
Bi-encoder y cross-encoder

Dos nombres nuevos para cosas conocidas. Ninguno apareció antes en el curso, y nombran exactamente los dos métodos de la lámina anterior.

Bi-encoder. Lo que hace SBERT y lo que usamos todo el día: cada texto se codifica por su cuenta, y la comparación es una operación geométrica entre los dos vectores. Barato, precomputable, escalable a millones — el pasaje se embebe una vez y se guarda para siempre.

Cross-encoder. El caro: la consulta y el pasaje entran juntos al modelo, que los lee como un solo texto y emite un puntaje. No hay nada que precomputar: hay que correrlo por cada par. Y puntúa mucho mejor, porque ve qué parte del pasaje responde a qué parte de la consulta.

La asimetría de información. El bi-encoder comprime el pasaje sin saber la consulta. El cross-encoder ve las dos cosas al mismo tiempo. No es que uno sea mejor modelo: tiene más información al decidir.



La diferencia no es la calidad del modelo: es qué sabe cuando decide. Comprimir sin conocer la pregunta es, necesariamente, comprimir a ciegas.

El caro vuelve, sobre cincuenta candidatos

El movimiento, en una frase. El cross-encoder era impagable sobre el corpus entero. Pero no hace falta: el recuperador ya redujo el problema a unas decenas. Cincuenta inferencias, no treinta mil.

El paradigma de dos etapas. Primera, barata y amplia: el bi-encoder y el canal léxico recorren el corpus y traen un conjunto generoso. Segunda, cara y precisa: el cross-encoder lee cada candidato junto con la consulta y los re-ordena.

Cambia cómo se elige k. Ahora son dos: cuántos se recuperan —generoso— y cuántos se entregan —chico, por el contexto—. Anthropic recupera ciento cincuenta y entrega veinte.

Por qué esto es afinar. No puede agregar nada que la primera etapa no haya traído: sólo reordena. Por eso va al final.

Y no es la única forma de rerankear. Lo define volver a mirar los candidatos con un criterio más caro. También se rerankea con un LLM que ordena la lista, con interacción tardía (ColBERT) y con APIs de pago: Cohere, Voyage, Jina.



Lo caro se vuelve barato cuando se lo corre sobre poco. No era el modelo el problema: era el tamaño del conjunto sobre el que lo corríamos. ▸ Notebook §11 — el mismo top-k antes y después de re-puntuar.

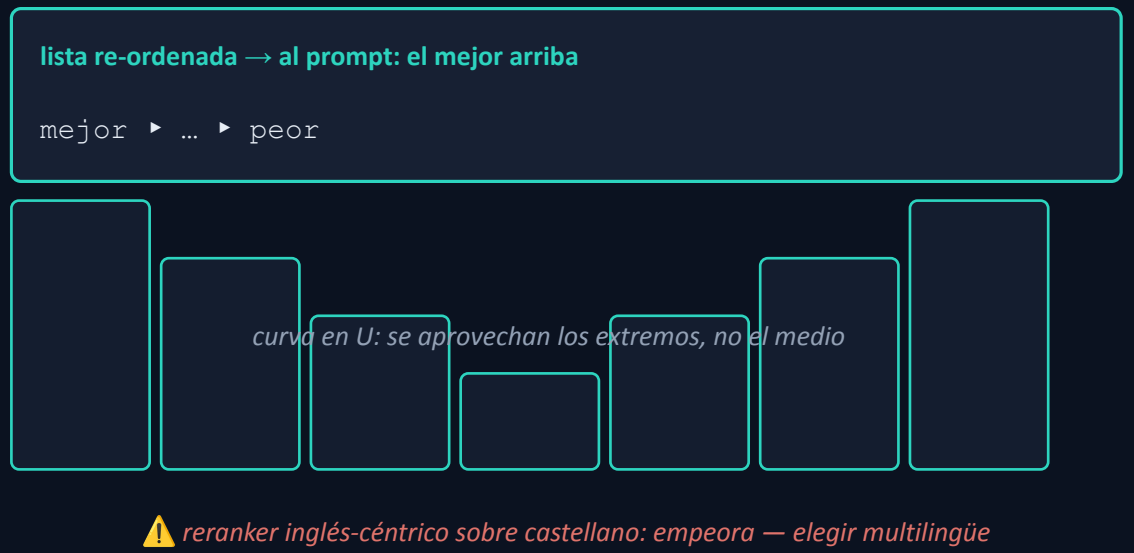
No sólo elige: produce el orden

Una deuda de prompt engineering. En prompt engineering vimos que la ventana de contexto no se usa pareja: el modelo aprovecha mejor lo que está al principio y al final, y peor lo enterrado en el medio. Y prometía que era input directo para RAG: dónde ubicar los pasajes.

Acá se cobra. El re-puntuador no devuelve un conjunto: devuelve una lista ordenada por relevancia real. Ese orden es el orden en que conviene pegar los pasajes, con el mejor donde el modelo mejor lo aprovecha. Elegir bien y ordenar bien son dos beneficios distintos.

La otra mitad del callout. Menos pasajes bien elegidos le ganan a muchos apilados. Con un re-puntuador confiable se puede bajar la cantidad entregada sin perder calidad: mejor cinco buenos ordenados que veinte sin ordenar.

El detalle que rompe todo. El re-puntuador también tiene que hablar el idioma del corpus. El modelo por defecto está entrenado sólo en inglés; sobre castellano puntúa mal, y el único movimiento de precisión empeora los resultados. Hay versiones multilingües, y hay que elegirlas a propósito.



Rerankear da dos cosas por el precio de una: cuáles se entregan y en qué orden. La segunda es gratis y casi nadie la aprovecha.

La pila completa

Las cuatro piezas no compiten: se encadenan. Cada una ataca una falla distinta y actúa en un momento distinto del recorrido. Puestas en orden, forman un solo camino.

El orden, y por qué es ese. Antes de indexar, el pedazo recibe su contexto. Al llegar la consulta, se busca por los dos canales y se fusionan por puesto. El conjunto ancho se re-puntúa. De lo que queda arriba, se entregan los pasajes grandes en el orden del re-puntuador. La misma pila que usa la industria.

La lógica es siempre la misma. Recuperá ancho, afiná después. Ensanchar primero porque lo que no entró no se recupera más; afinar después porque reordenar es barato cuando el conjunto ya es chico.

Lo que no está en esta pila. Nada de esto decide si hay que recuperar, ni reformula la búsqueda si el primer intento falló, ni recupera dos veces. Eso existe, tiene nombre, y son los escalones que dejamos marcados en el mapa del arranque.



Cuatro arreglos, cuatro fallas, un solo camino. Si alguna pieza no está resolviendo una falla que se mide, conviene sacarla: está costando latencia para nada.

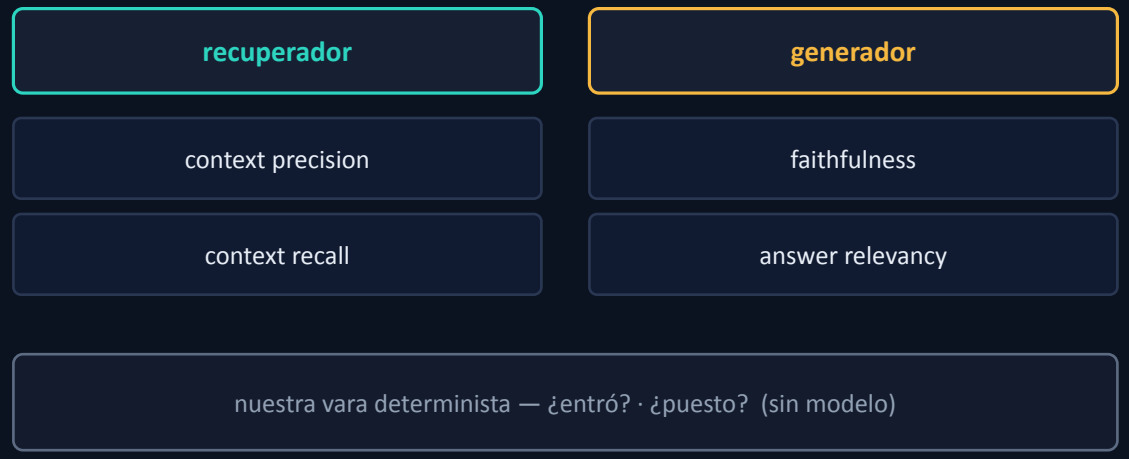
Esto está empaquetado, y se llama RAGAS

La vara que armamos alcanza para lo que hicimos. Mide si el documento correcto entró y en qué puesto quedó. Eso cubre la primera superficie de falla —la recuperación— y es lo que movieron las cuatro piezas.

Pero falta la otra superficie. ¿La respuesta se mantuvo fiel a lo que se le entregó? Eso no se mide contando documentos: hay que leer la respuesta, compararla con el contexto y juzgar. Es un trabajo de lectura, y por eso se automatiza con un modelo.

Hay una librería estándar. Se llama RAGAS y organiza la evaluación como la venimos pensando: métricas del lado del recuperador y del lado del generador. Las cuatro por defecto son context precision, context recall, faithfulness y answer relevancy.

Cómo la usamos hoy. Con resultados precomputados sobre pocas preguntas. No porque no corra, sino porque corre lento con un modelo local y porque su valor hoy es conceptual: entender qué mide cada una y qué NO mide.



La estructura de la librería es la de la clase: dos lados, dos preguntas. Eso es lo que hay que llevarse — más que los nombres de las métricas, que cambian entre versiones.

Las cuatro métricas, y todas usan un juez

El dato antes de correrla. Las cuatro métricas por defecto invocan a un modelo. No es que faithfulness use juez y las demás sean cálculos: las cuatro son juicios automatizados, y por eso una evaluación de cincuenta preguntas son cientos de llamadas.

Qué hace cada una. Faithfulness descompone la respuesta en afirmaciones y verifica cada una contra el contexto. Answer relevancy genera preguntas a partir de la respuesta y mide si se parecen a la original. Context precision juzga relevancia por cada pasaje. Context recall parte una respuesta de referencia y chequea qué cubre el contexto.

Lo que implica para el eval set. Context recall necesita una respuesta de referencia escrita de antemano —no alcanza con la pregunta—. Es la más cara de preparar, y la que menos se parece a lo que nuestra vara ya medía.

Y existen variantes sin modelo. La librería tiene versiones que comparan por identificador o por distancia de strings, sin juez. Son, esencialmente, lo que hicimos a mano hace cuarenta minutos — pero no las que corren por defecto.



context recall necesita respuesta de referencia · las variantes sin juez no son el default

Antes de correr una librería de evaluación, conviene preguntarse cuántas llamadas son. Cuatro métricas por muestra, varias llamadas por métrica. Eso decide si se pueden evaluar cincuenta preguntas o cinco.

Dos líneas de honestidad

Primera: esto es el juez que vimos en alineamiento y evaluación. RAGAS es LLM-as-a-judge aplicado a RAG, y hereda todos los sesgos que desarmamos a mano al tratar alineamiento y evaluación: prefiere respuestas largas, prefiere su propio estilo, se deja llevar por el orden. Un número de un juez sesgado sigue siendo sesgado.

La trampa de Goodhart, otra vez. Si se optimiza el pipeline para que la métrica suba, y la métrica es un juez con sesgo de verbosidad, se termina con un sistema que escribe más largo. La medida, vuelta objetivo, deja de medir.

Segunda: faithfulness alto ≠ correcto. Significa respuesta fiel al contexto. Si se recuperó basura y el modelo fue fiel a la basura, la métrica da un número alto — es la celda "fiel a basura" de la matriz, ahora medida. Faithfulness mide consistencia con lo entregado, no verdad.

Por eso las dos superficies juntas. Faithfulness alto con recuperación mala es un sistema que miente con coherencia. Sin la métrica de recuperación al lado, ese caso se lee como éxito.



La métrica dice qué pasó, no si estuvo bien. Un número alto en una sola superficie puede ser exactamente el peor caso. Por eso hay dos.

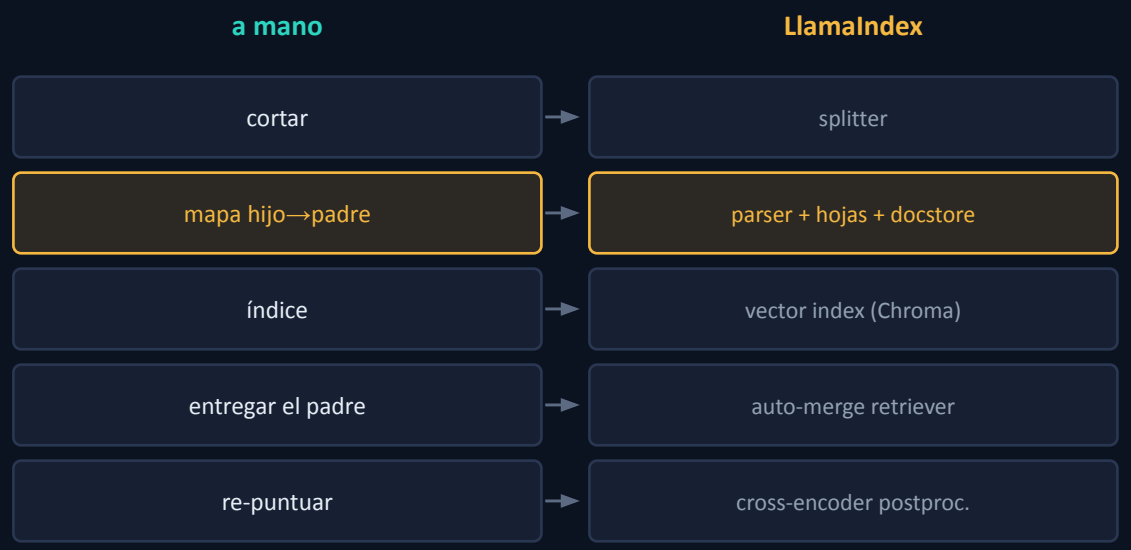
El mismo pipeline, en un framework

Todo lo de hoy tiene nombre en LlamaIndex. No hay magia nueva: las mismas piezas, empaquetadas, con la configuración expuesta. Es el mismo movimiento que hizo el curso con PyTorch después de construir el motor de derivadas a mano.

El mapeo, pieza por pieza. Cortar es un splitter. La jerarquía padre/hijo es un parser jerárquico, más quedarse con las hojas, más un almacén de documentos. El índice sobre Chroma es un índice vectorial. Recuperar el hijo y entregar el padre es un recuperador con auto-merge. Re-puntuar es un post-procesador de cross-encoder.

El detalle que hace honesta la comparación. El auto-merge no funciona solo: necesita haber parseado jerárquicamente, indexado sólo las hojas y declarado el store. Lo que a mano era un diccionario, el framework obliga a declararlo como componente. Gana robustez, pierde transparencia.

Una trampa para el lab. Por defecto apunta a modelos de pago: si no se declara qué modelo y qué embedder usar, la primera celda pide una clave de API. En un curso sin APIs pagas, eso es lo primero que hay que configurar.



Lo que se construyó a mano tiene nombre propio en el framework. Y el framework obliga a declarar lo que a mano era un diccionario. Por eso se construye primero a mano. ▸ Notebook §12 — el pipeline en LlamaIndex.

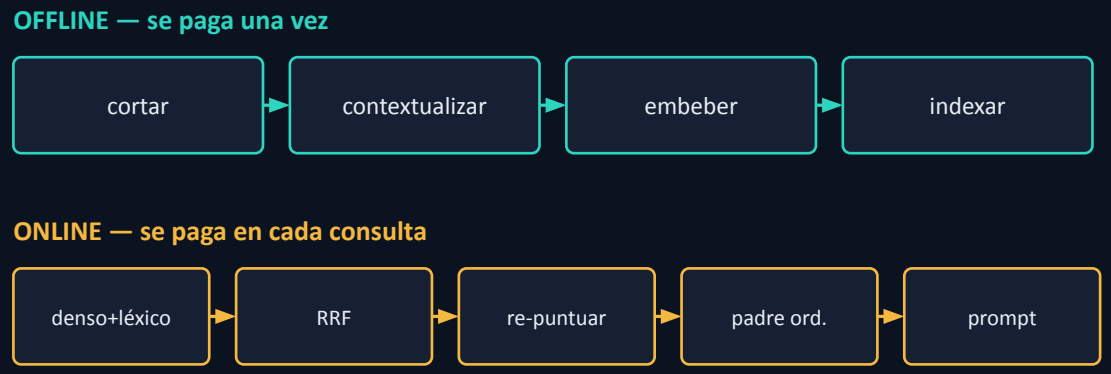
El pipeline final

Las dos fases, con todo adentro. Offline: cargar, cortar por estructura con solapamiento, contextualizar cada pedazo, embeber e indexar con metadatos. Online: embeber la consulta, buscar por los dos canales, fusionar por puesto, re-puntuar, entregar los padres en orden, y armar un prompt con contexto delimitado, abstención y cita.

Cada pieza, con su falla al lado. Ninguna está porque sí: el solapamiento por el corte que parte una oración, la contextualización por el pedazo huérfano, el canal léxico por el identificador, el re-puntuador por el ruido, el padre por el fragmento sin contexto, la abstención por la respuesta inventada.

Y con su costo. Las offline se pagan una vez; las online, en cada consulta. Un pipeline que no puede justificar cada pieza con una falla medida está pagando latencia de más.

Esto es lo que se lleva a un proyecto. No la lista de técnicas: el criterio para decidir cuáles poner, en qué orden, y cómo saber si sirvieron.



un pipeline es una lista de fallas resueltas

Un pipeline es una lista de fallas resueltas, no una lista de técnicas aplicadas. Si no se puede nombrar la falla que motivó una pieza, esa pieza sobra.

CIERRE · LA SÍNTESIS

Nunca tocamos los pesos

El modelo de hoy es el mismo que usamos en prompt engineering. No lo entrenamos, no lo ajustamos, no lo cambiamos. Y sin embargo responde cosas que antes no podía, sobre documentos que nunca vio en su entrenamiento.

El contraste vale decirlo. La segunda unidad del curso fue enteramente sobre tocar los pesos: preentrenamiento, fine-tuning, alineamiento. Todo lo de hoy consiguió un cambio de capacidad práctica sin tocar uno solo. Todo fue ingeniería de qué lee el modelo.

La idea que ordena todo. Lo que se recupera es el techo de lo que la respuesta puede ser. Por eso invertimos la clase en traer el pasaje correcto, y por eso las técnicas inconexas eran la misma: chunking, indexado, canal léxico, contextualización, re-puntuado, transformación de la consulta. Una pregunta, seis lugares donde atacarla.

Con su matiz. El techo es una cota superior, no una garantía: la generación decide cuánto se alcanza, y con un modelo chico alcanza menos. Por eso el prompt de RAG era parte de la clase.



La recuperación fija el techo; la generación decide cuánto se alcanza. Esa es la clase entera, y es el criterio con el que se diagnostica un RAG que anda mal.

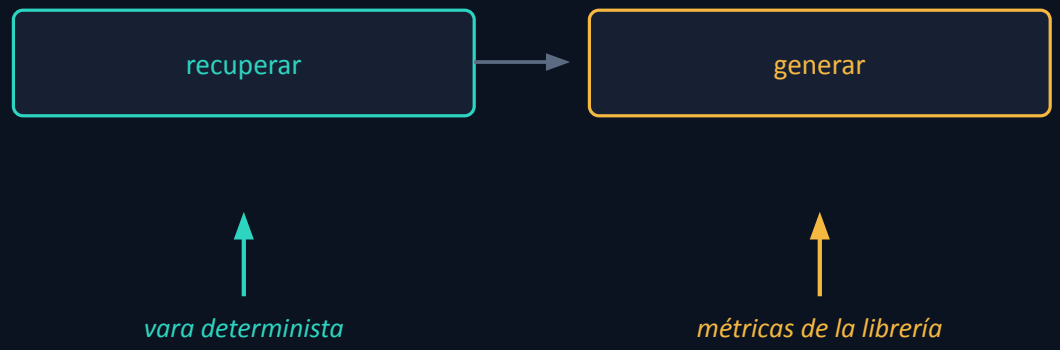
Qué mide, y qué no mide, lo que medimos

Sería lindo decir que la evaluación mide lo que movimos. No es así, y conviene terminar con la versión correcta en vez de con la simétrica.

Lo que sí es cierto. Las métricas cubren las mismas dos superficies de falla que atacó la clase: la recuperación, a la que dedicamos casi toda la segunda mitad, y la generación, que atacamos con el prompt de RAG al cierre de la primera.

Lo que no es cierto. Que los números midan las mismas nociones que las palabras. Las métricas del recuperador miran la lista final, la que llega al generador — no el conjunto ancho intermedio que ensanchamos. Y context recall, pese al nombre, se computa contra una respuesta de referencia.

La conclusión práctica. El pipeline tiene dos etapas, así que una sola métrica al final no dice cuál falló. Por eso armamos la vara determinista sobre la recuperación: cuando el número global baja, hay que poder mirar cada etapa por separado.



mismas superficies, distintos números — medí cada etapa

Medir cada etapa por separado; si no, es adivinar. Es la misma razón por la que separamos las dos superficies de falla al final de la primera mitad.

CIERRE · EL HILO CON EL CURSO

El hilo con el resto del curso

Hacia atrás. Los sentence embeddings se volvieron el motor de recuperación, y su invariante se cobró al cambiar de embedder. La tokenización decidió el tamaño de los pedazos. La alucinación, del lado del alineamiento, fue el problema de partida, y su juez volvió como instrumento. El contexto finito de prompt engineering fijó el presupuesto, y su curva en U decidió el orden.

Hacia adelante, tres piezas. Hoy el sistema no recuerda nada entre preguntas: darle memoria es la clase de memoria y gestión del contexto. Hoy recuperamos sobre texto plano; recuperar sobre información estructurada —entidades y relaciones— es la clase de Graph RAG. Y hoy el pipeline es fijo: siempre recupera, siempre igual.

La que cierra la unidad. Un sistema que decide por su cuenta si necesita recuperar, qué consulta emitir y si lo que trajo alcanza, es un agente usando la recuperación como herramienta. Eso es agentic RAG, el escalón que quedó marcado en el mapa del arranque.

Y una frontera que hay que nombrar. Todo lo que entra al índice entra, tarde o temprano, al prompt. Un pasaje puede contener instrucciones dirigidas al modelo, y el sistema no tiene cómo distinguirlas del contenido. Es la primera vez que texto de terceros llega al prompt.



Hoy le dimos al modelo acceso a algo fuera de sí mismo. Es el primer paso de la segunda mitad del curso: de un modelo que responde, a un sistema que consulta, recuerda y actúa.

Cierre y recursos

Lo que exhibe el notebook. Corre el pipeline entero sobre el corpus: el baseline de embeddings, el A/B de tamaños de pedazo con el acierto al lado, el índice plano contra el store, el canal léxico y la fusión por puesto a mano, el mapa hijo→padre, el prompt con abstención y cita, la tabla del eval set, y la misma pila en LlamaIndex. En clase se ven salidas capturadas.

Lo que sigue. La próxima clase, memoria y gestión del contexto: qué hacer cuando la ventana se llena, y cómo un sistema recuerda entre una pregunta y la siguiente.

Núcleo Lewis 2020 (RAG original) · Gao 2023 (survey, el mapa) · Anthropic 2024 (Contextual Retrieval, con sus caveats)

Para afianzar Gao 2022 (HyDE) · Cormack 2009 (RRF) · Nogueira & Cho 2019 (cross-encoder) · Robertson & Zaragoza 2009 (BM25) · Malkov & Yashunin 2020 (HNSW) · Liu 2023 (Lost in the Middle) · docs de RAGAS

Por qué no fine-tunear hechos Gekhman 2024 · Ovadia 2023 (Fine-Tuning or Retrieval?)

Práctica docs de LlamaIndex, Chroma, sentence-transformers · DeepLearning.AI (Advanced RAG)

Fuente de un hilo de hoy — las charlas de Andrej Karpathy: el contexto como memoria de trabajo del modelo, y los recursos externos como el almacenamiento del que se trae lo que hace falta. El puente: con el modelo capaz de consultar una fuente, lo que sigue es que recuerde. ▸ Memoria.